Client-Server Texas Hold 'Em

Senior Project Report

Andrew Matthews

Table of Contents

Background

My project is a Client-Server Texas Hold 'Em card game with an AI component. A first player will run the server-side application in which they can choose for three other players to be either AI or client players. The server-side application will then wait until all clients have connected before starting the game. On the client-side application, upon starting the application, the player will have to enter the IP address of the server-side application. The port number is hard coded into both sides as 3000, and the server-side application is configured to accept any IP address trying to connect. As I mentioned, once the number of clients to connect is equal to the amount specified at the start screen, the game will then proceed for all parties.

The rules of Texas Hold Em' are similar to rules of any poker game. At the beginning of the game, each player is dealt two cards and five cards are placed face down in the center; these cards are called community cards. The object of the game is to have the best five card hand combination of the seven possible cards, the possible cards being the two cards unique to each player and the community cards after having been flipped. An initial betting round ensues where players are betting solely on the two cards in their hand. Players can either raise the amount of chips in the pot, call an amount of chips if they have less in the pot than everyone else, check if they have the same amount of chips in the pot as everyone else, or fold. The betting will continue for one cycle through all the players, and then until all players have the same amount of chips in the pot. After the first betting round, three of the community cards will be exposed. This is called the flop. Another betting round takes place after the flop. Two more rounds occur where the fourth and fifth community cards are exposed followed by a round of betting. These rounds are called the turn and the river, respectively. In the final stage, the showdown, the player with the best hand is awarded all the chips in the pot.

The game itself, at least graphically, is similar to an old windows card game; it has a green background with simple card images. The window has a fixed size and each player has their cards facing the center from each side. In the middle of the table, as you would probably suspect, are the five community cards. The interface provides a button for each move you would associate with poker, i.e., fold, call/check, and raise. There is a list box that tracks player moves and game events, and will signify when it is your turn. For each player, it shows their name and the amount of chips they have. There is also a label for the amount of chips currently in the pot.

The client-server communication utilizes the TCP Client class which is part of the .net framework. Using this class abstracts a lot of what is going on behind the scenes and makes it much easier to implement network communication between applications. Calling Stream.Read(bytes, 0, bytes.length) will read what is currently in the TCP Client stream. This is not guaranteed to be a certain number of bytes though. For this reason, it is important to either send and receive data in fixed sizes, or delimit all messages. In my case, I chose to delimit my strings with the signifier <end>. The read function is then placed in a loop that will only break if it finds the delimiter. The data that is received is then processed. On the client side this message is either "isturn," which lets the client player know that it is now their turn, or a serialized BoardState object, which stores information on the state of the playing board, i. e., their cards, other player's cards, community cards, list box messages, chip amounts, and the pot amount. After every move, the server-side application will broadcast the board state to all clients which will update their interface.

AI

The AI players are mainly configured to play the odds with some degree of randomness provided. At the beginning of each game there is a 10% chance of a boolean variable, Bluff, being assigned to true, with the default value being false. If Bluff is true for a player, whenever the TakeTurn algorithm determines that a player should call, check, or fold, the player will instead raise a random amount based on the amount of chips they have. If Bluff isn't true, the first thing to consider is the betting round. If it is the final betting round there is no need to calculate outs because there are no more cards to factor in. The pre-flop betting round is also a special case. The betting rounds after the flop and turn can be treated as the same with the exception that the odds to get outs will vary because of the number of unexposed community cards left. The first thing my AI player will do is check to find his current best hand. The current hand rank will be an integer on a scale of 0 to 100 with 0 being a high card of less than 5 and 100 being a royal flush. The rank has a starting point for the hand itself, and an integer is added to it based on the high cards. This is provided that the betting round is post-flop; if it is pre-flop, he will only have two cards which are not enough to make a hand. Immediately following the flop, the player will have enough cards to make a hand, though at this point there is no optimal hand, only the hand he has. The two following betting rounds he will find and use the optimal hand, however. After this, if it is not the final round, he will calculate the outs. In poker, an out is any card that will give you a winning hand. This is an inexact science as you can't define a winning hand, at least not until the showdown. However improbable, it is still possible you can lose with a straight or a flush. To calculate outs, I determined if the hand was one card from completing any hand that was a straight or better. Along with the straight this includes: Straight flush, four of a kind, full house, and flush. You may have outs for more than one hand; the more outs you have, the better your chances

are. The AI then compares a combination of the current best hand rank and the likelihood of the next cards being outs to the pot odds, which is basically a ratio of the amount you would have to call to the amount currently in the pot. If the player has a straight or better and the value calculated from determining the outs, which is also on a scale of 0 to 100, is less than the hand rank value, meaning there isn't great potential for a better hand, then the outs value is replaced by the hand rank value. These values are then added together. Next, a random number is generated from 1 to 100. If this number is less than the rank number, the player will raise. The amount to raise is random, but the range is determined by the rank as a decimal multiplied by the number of chips the player has. If the player is able to check, he will be determining whether he should raise or not based on his hand and potential hand. For this reason, he will not have to compare his rank to the pot odds. If the round is pre-flop, the players betting will be based on whether he has a pair or not, whether his cards are of the same suit, and his combined high card value.

What I Learned

I learned a number of things in the process of creating this project. The first thing I had to make myself familiar with was the language of C#. I had some experience with it and I knew it was similar syntactically to Java, but with any language it's going to have caveats and additions that make it different. For me personally, C#, in conjunction with the .net framework, is probably now my favorite language. An example of one of the cool things you can do with C# would be lambda expressions like this one

```
hcs.Exists(c => c.CardType == CardType.Ace)
```

which returns true if an ace is found (hcs is a list container of type Card). This is just one example of the freedom you have with the C# language.

Beyond the language itself I learned a great deal about certain concepts. One such concept is event-driven programming. Event-driven programming is a programming paradigm in which the flow of the program is determined by events. In my case this was button click events for certain actions. Much of this can be automated by Visual Studio; you can click a button twice and Visual Studio will create a button click event and event handler. Event-driven programming was a new way of thinking for me. Initially, I was inclined to have the game play code in a loop, but I couldn't pause the loop to wait for user input when it was their turn (This may be possible, but it would be very poor design).

Serialization was also a concept that was fairly new to me. Serialization is the process of converting an object state into a format that can be stored and restored later in another computer environment. There are a number of different serialization methods that are available: XML serialization, binary serialization, SOAP serialization, and other .net implementations (protobuf-net). XML serialization will embed the object's state in an XML stream. Binary serialization will create a binary stream from the object's state. SOAP is a

protocol based on XML, designed specifically to transport procedure calls using XML. Protobuf is an implementation specific to .net that has been lauded for its simplicity and speed. In benchmark tests that can be found online, BinaryFormatter, the .net default for binary serialization, was consistently outperformed by XmlSerializer, the .net default for XML serialization. Although it wasn't necessary for my project, XML serialization is also advantageous because of its interoperability; many binary serializers, including BinaryFormatter, are not. For these reasons, I ultimately chose XML serialization which served my project well. The following is an example of an XML serialized BoardState object

```
<?xml version="1.0" encoding="utf-16" ?>
 <BoardState xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
        <CommunityCards>
                <ShellCard xsi:nil="true" />
                <ShellCard xsi:nil="true" />
                <ShellCard xsi:nil="true" />
                <ShellCard xsi:nil="true" />
                <ShellCard xsi:nil="true" />
        </CommunityCards>
        <PlayerCards>
                <ShellCard>
                        <Suit>Diamonds</Suit>
                        <CardType>Nine</CardType>
                </ShellCard>
                <ShellCard>
                        <Suit>Clubs</Suit>
                        <CardType>Six</CardType>
                </ShellCard>
        </PlayerCards>
        <Players>
                 <ShellPlayer>
                        <Hand>
                                <ShellCard>
                                        <Suit>Spades</Suit>
                                        <CardType>Eight</CardType>
                                </ShellCard>
                                <ShellCard>
```

```
                    <Suit>Hearts</Suit>
                    <CardType>Queen</CardType>
                </ShellCard>
            </Hand>
            <StillIn>true</StillIn>
            <Chips>200</Chips>
            <ChipsInRound>0</ChipsInRound>
            <Name>Andrew</Name>
        </ShellPlayer>
        <ShellPlayer>
            <Hand>
                <ShellCard>
                    <Suit>Hearts</Suit>
                    <CardType>Nine</CardType>
                </ShellCard>
                <ShellCard>
                    <Suit>Spades</Suit>
                    <CardType>King</CardType>
                </ShellCard>
            </Hand>
            <StillIn>true</StillIn>
            <Chips>200</Chips>
            <ChipsInRound>0</ChipsInRound>
            <Name>Dave</Name>
        </ShellPlayer>
        <ShellPlayer>
            <Hand>
                <ShellCard>
                    <Suit>Hearts</Suit>
                    <CardType>Six</CardType>
                </ShellCard>
                <ShellCard>
                    <Suit>Spades</Suit>
                    <CardType>Seven</CardType>
                </ShellCard>
            </Hand>
            <StillIn>true</StillIn>
            <Chips>200</Chips>
            <ChipsInRound>0</ChipsInRound>
            <Name>Tony</Name>
        </ShellPlayer>
</Players>
```

```
        <PotAmount>0</PotAmount>
        <CallCheckMessage>Call 5</CallCheckMessage>
        <MoveMessage />
        <Chips>200</Chips>
</BoardState>
```

This XML string is then deserialized on the client side to a restored BoardState object. XML serialization is simple enough in C#. You just have to make sure the class in question is XML serializable. This is as simple as adding a [serializable] attribute to your class as long as all types within your class are also serializable. There are ways to customize the XML, but in my case the default worked just fine.

In my project I was also able to get my feet wet with artificial intelligence programming. As far as artificial intelligence goes, developing an AI player for a card game is pretty basic, but it was still a beneficial experience for me, and certainly I could have made it more complex had I not run out of time. With a game like mine, it is probably best for the user experience that the AI is not fully optimal if such a thing exists. An optimal AI would most likely be predictable and I believe a certain amount of randomness is important to the user experience. One thing I wanted to implement in the AI, but didn't in the end was to look for patterns in other player's behavior, i. e., whether they bluff often, or only raise if they have a good hand. My suspicion is that this would not make the AI much smarter unless done perfectly. If the AI's decisions are weighted too heavily on what other players have done in the past, there is a risk of being too quick to fold or call a bluff. Still, this would have been something fun to experiment with had I had more time.

I also was able to learn about client-server programming. This is a concept not entirely new to me as it is a subject introduced in Applied Networking and briefly covered in a few other classes here at CSU. Knowing what a client and server are is very different than being

able to integrate the concepts programmatically however. As I mentioned earlier, the TcpClient class makes things fairly painless though. Once you can establish a connection the final step is to use the stream's read and write methods to establish a protocol that is specific to your application.

Problems faced

Unfortunately, not everything I programmed worked the first time. As is often the case, I had to experiment and research problems to find solutions. As I covered earlier, when I was working on the basic game play I initially wanted everything in a loop. This wouldn't work because I needed the program to stop execution and wait for user input. The simple solution was to refactor my code to fit the event-driven programming paradigm.

One problem I experienced from the very beginning that turned out to have the easiest solution of all was that of flickering graphics. After every turn the cards would flicker as they had to be repainted. The solution is to double buffer and it turns out that the Form class has property DoubleBuffered, which, if set to true, will solve this problem for you.

I ran into quite a few hiccups when trying to serialize an object. I initially tried to use binary serialization, but I was not able to get it to work. The root of my problem had to do with encoding of the message string. My client code was reading into an ASCII string to find the delimiter, but the binary deserialization called for a base 64 string. For some reason, I could not get any conversion to work. Since there is no performance drop-off with XML serialization, I then decided to use XML serialization. The next problem was that I had created separate BoardState classes for the Client and Server applications. As a result, the serializer could not find the appropriate assembly to deserialize. The solution was simply to share the assembly between the two projects. This solved that problem, but unfortunately it was not the end of my issues. I was still getting an error when trying to deserialize that didn't give me any specifics. I was able to determine that the card Images, which are part of the player class, were not serializing correctly. The solution to this problem was to retrieve the image from the client based on the card type and suit.

Possible Extensions/Improvements

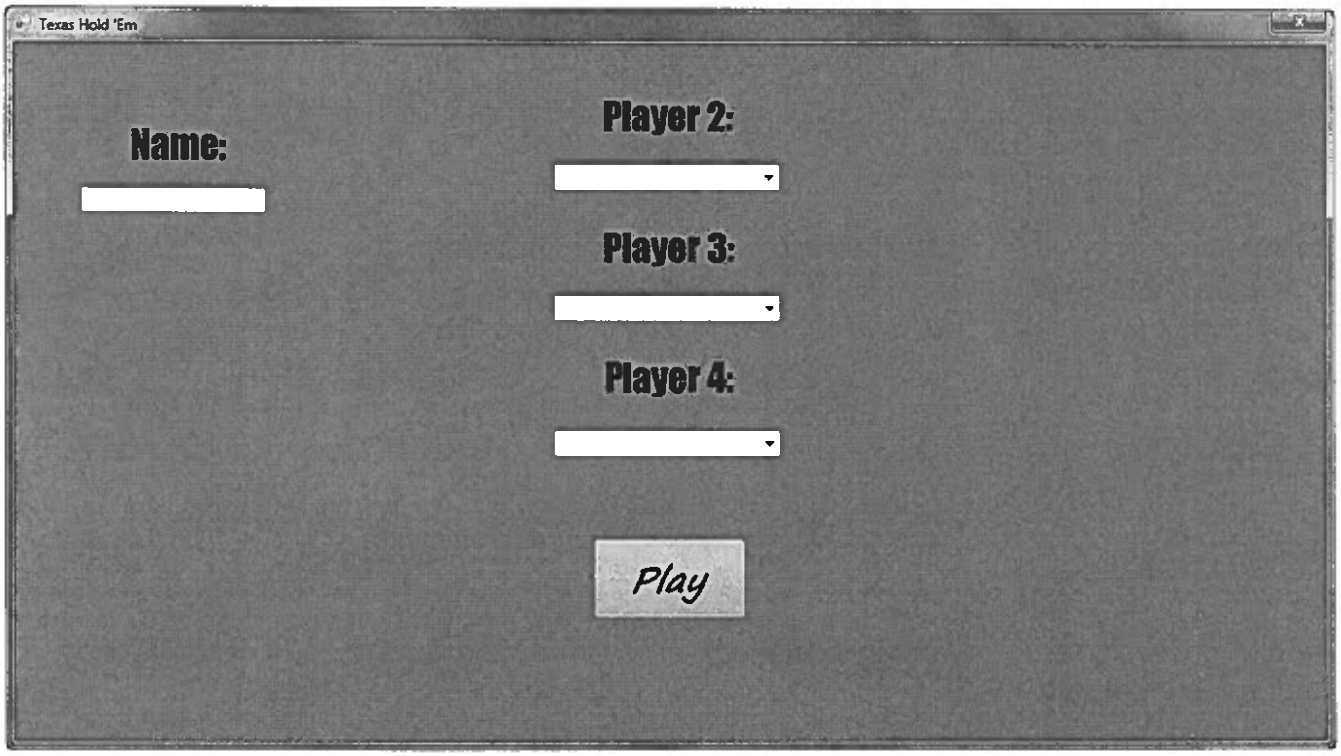
Although I am fairly satisfied with the outcome of my project, I do not consider it to be a finished product. There are still a number of things I would implement if I had more time. As I touched on earlier, I feel that the AI can be expanded upon. The first thing I would add is pretty simple: Have different difficulty levels of AI to be chosen at the initial screen. I could do this fairly easily by increasing randomness for easy players and having harder players play the odds pretty strictly. Another thing that wouldn't be immediately obvious is to weight the rank of a hand based on how many of the cards are part of the community cards. I think this would be a good idea because the more cards you have out of your best hand that are part of the community cards, the more likely an opponent has a similar or better hand. For example, if you have a three of a kind with two of the three cards in the community cards, and your two cards do not have high ranks, you may want to tread cautiously as there is a decent chance that someone else may have a three of a kind with better high cards. The final thing I would look into implementing if I could would be having the AI look for certain patterns or tendencies in other players. I mentioned this earlier, but if a player has a tendency to bluff a lot or only keep playing if they have a good hand, then this information can be used to help determine their moves.

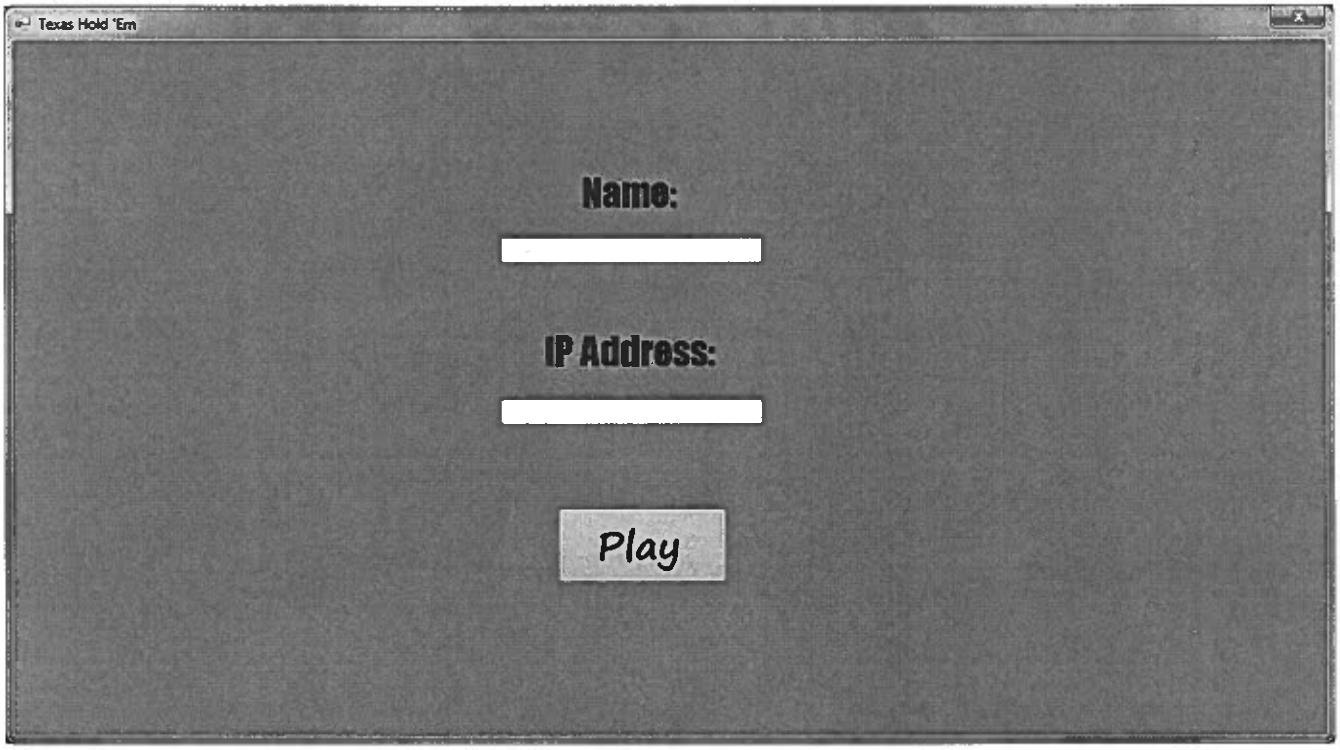
Another thing that I think would really improve my project would be to have a way to save a player's progress. This would be as simple as saving a name and a chip amount to a database or file. The game would have to be updated in other ways though; you would have to allow for the option of choosing an existing user player.

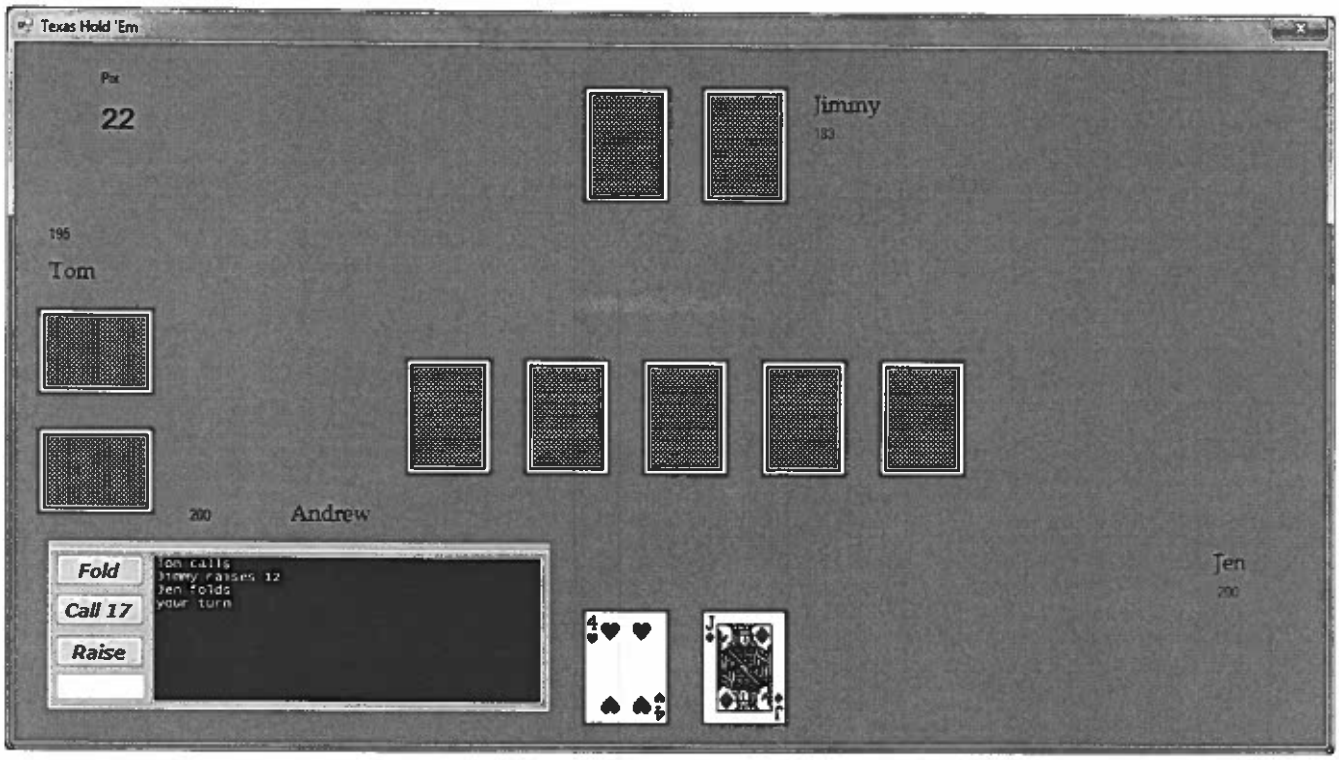
The final thing I would add, that I have thought of, would be a waiting screen or message for the server-side user to know that they are waiting for clients to connect. This one is pretty obvious when playing the game. If you select any of the players to be clients, when

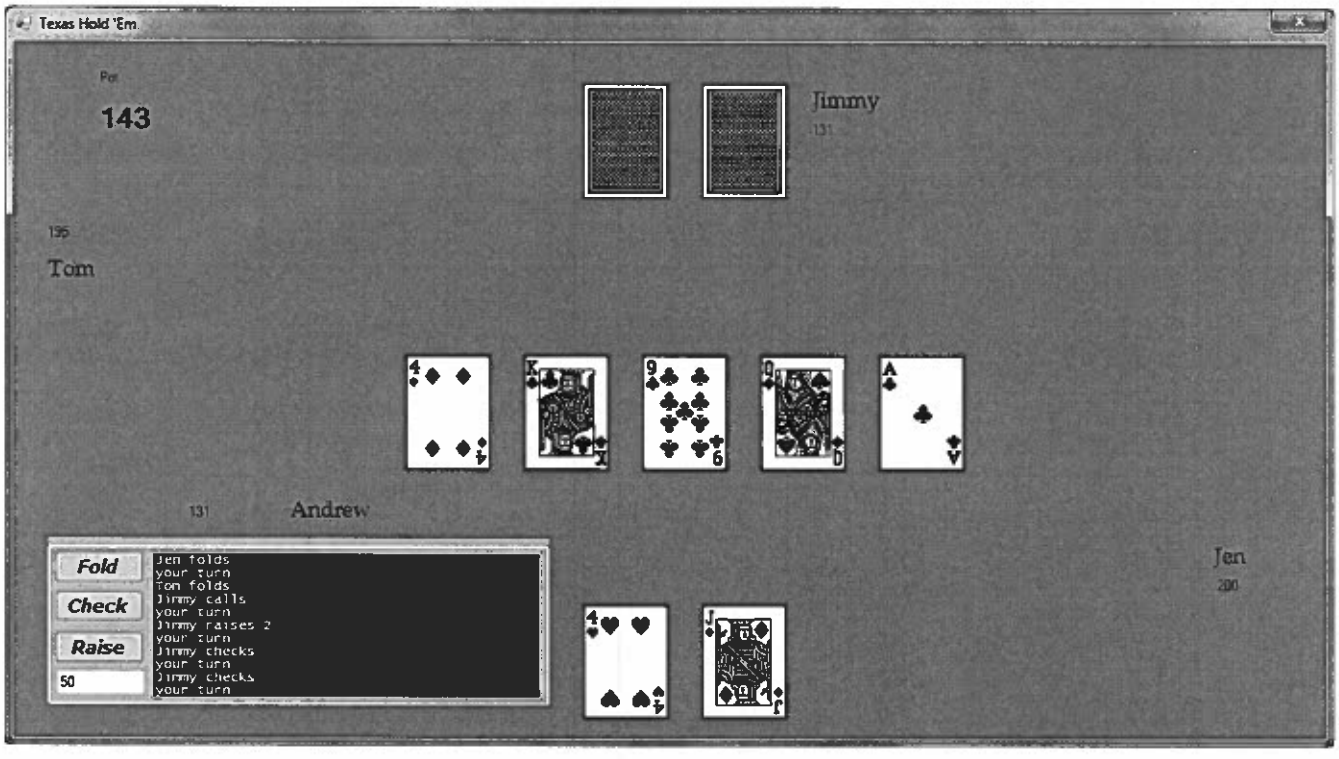
you hit play the screen does not change until all clients have connected. If you didn't know any better you may think the game had frozen. You also lose control of the user interface during this time which makes it seem even more like it may be frozen. This is an issue I have throughout the game that is most likely a design flaw. All of my code is run on the UI thread, so if you occupy the thread, the user is not able to use the interface. In these instances the player is not allowed to do anything using the interface (call/check/raise/fold) anyway, but ideally they should at least be able to drag the window around.

Images


Texas Hold 'Em
Name:
Player 2:
Player 3:
Player 4:
Play

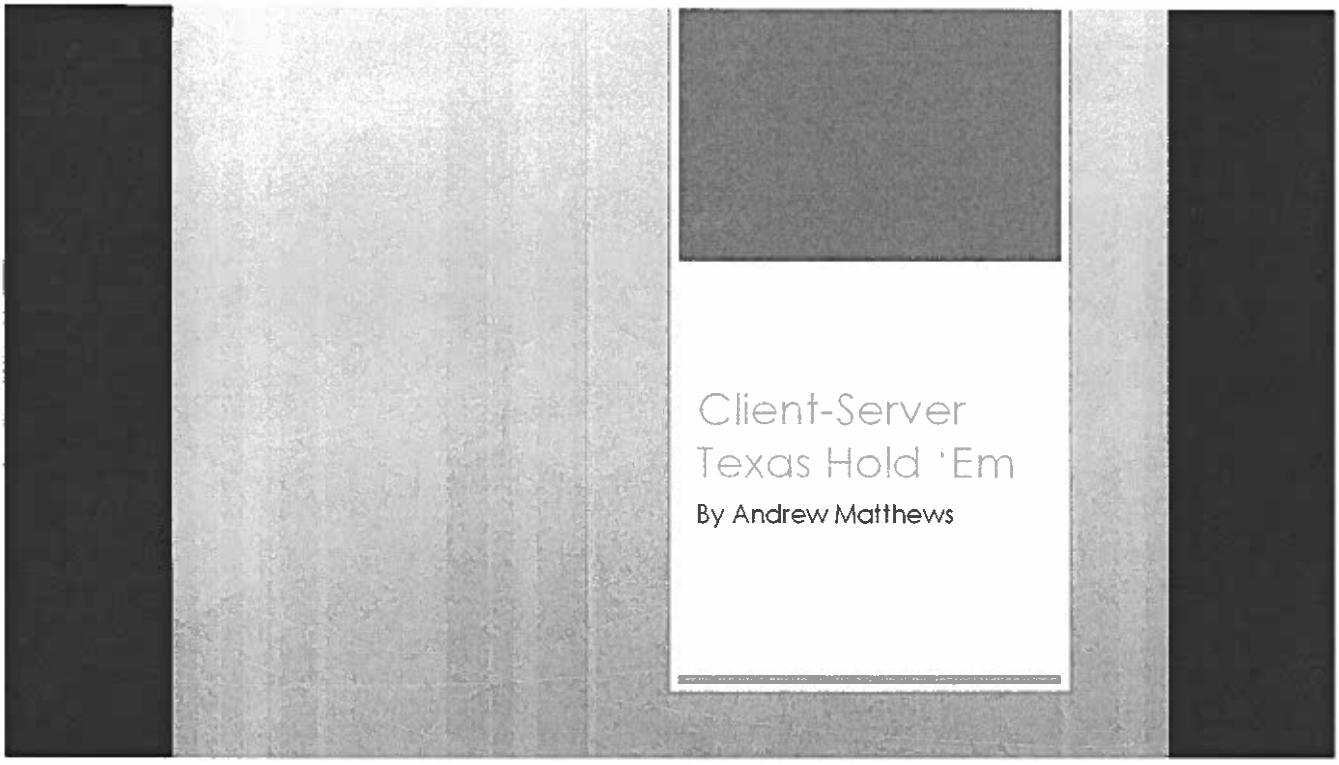

Texas Hold 'Em
Name:
IP Address:
Play

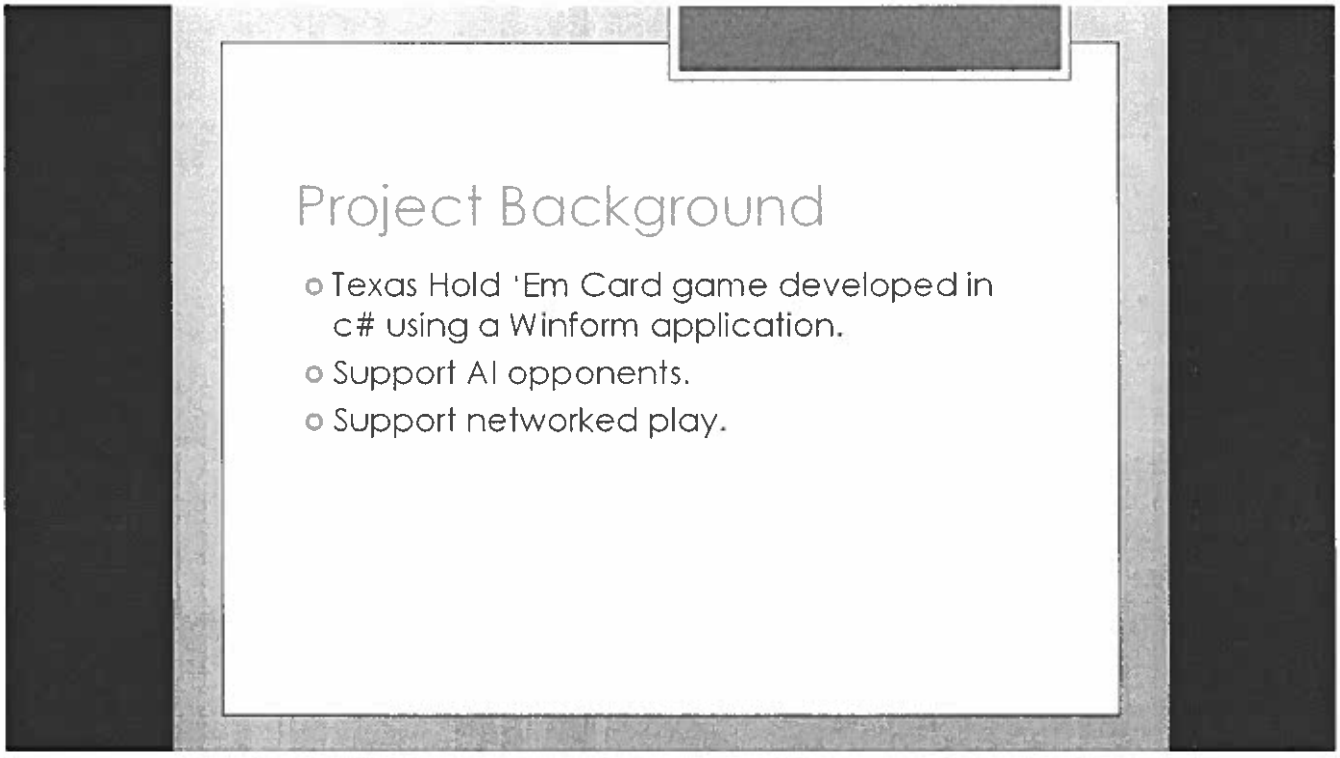

Texas Hold 'Em
Pot
22
Jimmy
183
195
Tom
200
Andrew
Jen
200
Fold
Call 17
Raise
Ton calls
Jimmy raises 12
Jen folds
your turn


Texas Hold 'Em
Pot
143
Jimmy
131
195
Tom
131
Andrew
Jen
200
Fold
Check
Raise
50
Jen folds
your turn
Ton folds
Jimmy calls
your turn
Jimmy raises 2
your turn
Jimmy checks
your turn
Jimmy checks
your turn

How to use

Stored on the DVD, I have the executables for both programs in their respective folders, THEServerEXE and THEClientEXE. I also included the Visual Studio Solution. I developed my game in Visual Studio Express 2012 for Windows Desktop, so any other version of Visual Studio may have some compatibility problems. The only other problem I can think of for recompilation is that I hard coded some full paths into my game. This would have to be changed before compiling. It is also important to disable the firewall when playing the game to allow network communication. When playing outside of the Visual Studio environment I experience some problems. For some reason, you have to immediately connect to the server. It seems like a timeout of some kind, but I don't know why I wouldn't experience this in Visual Studio. What you have to do is have the client ready to connect so you can hit play for the client right after you hit play for the server. Occasionally the window will stop responding momentarily. It is not consistent, so I'm not sure what the problem is. I do not experience any of these problems when building and running from within Visual Studio, so I would recommend doing that. Also, if testing the client and server application on the same machine, you can use the loopback IP address 127.0.0.1.


Client-Server
Texas Hold 'Em
By Andrew Matthews


Project Background
Texas Hold 'Em Card game developed in c# using a Winform application.
Support AI opponents.
Support networked play.

Basic Gameplay

- Understanding the rules
- Event driven programming

AI

- The most important thing is to rank current hand.
  - Straight Flush
  - Four of a kind
  - Full house
  - Flush
  - Straight
  - Three of a kind
  - Two pair
  - One pair
  - High card

AI contd.

- The second most important thing to consider is the number of outs.
- Outs are the cards it takes to give you a winning hand.
- Ex. – If after the flop you have 4 cards out of 5 necessary to make a straight, then the outs percentage would be 4(for each suit)/41 (cards left in deck) for next card.

AI contd.

- Lastly, combine current hand rank and outs and compare to pot odds.
- Compare the amount you would have to call vs. the amount in the pot already. Compare this to chance of winning.
- Ex. – If you have to call 10 versus 200 already in the pot and you calculate you have a 30 percent chance of winning 10/200 < 30/100.

AI contd.

- Make decisions based on a random factor.
- Ex. – if AI determines it should fold, give it a 20% of calling anyways and vice versa.
- An optimal AI is not necessarily the best for user experience.

Networking

- Use .net functionality – tcp socket programming.

Networking contd.

- Pitfall to avoid – attach delimiter to data or send data chunks of the same size back and forth.
- Stream.Read(bytes, 0, length) is not guaranteed to have all of the bytes you send on the first read.
- I attached the delimiter "<end>" to the end of my strings.

Networking contd.

- Serialize (from server) and deserialize (on client) the state of the playing board to send to clients.
  - Xml serialization
  - Binary serialization
  - Soap serialization

Challenges

- Making AI not predictable or super conservative.

Challenges contd.

- Understanding socket programming.

Challenges contd.

- Understanding serialization.
- Xml serialization vs. binary serialization.

Challenges contd.

- Thinking in terms of event driven programming instead of putting gameplay code in a loop.

Unfinished Business / Bugs

- Cards occasionally show up sideways. Image has to be rotated for certain players and it is not rotated back.
- Game does not have a smooth ending. Exiting one program prematurely will crash the other.

Demonstration

Source Code

THEServer

Form1.cs

```
using System;

using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using System.Threading;
using System.Threading.Tasks;
using SharedAssemblies;

namespace THEServer
{
    public partial class Form1 : Form
    {
        Size size, size2;
        Point[] myPoints;
        Point[] player2Points;
        Point[] player3Points;
        Point[] player4Points;
        Point[] communityCardsPoints;
        Bitmap horizontalCard, verticalCard;
        Deck deck;
        Card[] communityCards;
        bool first, fullCircle;
        bool haveClients;
        List<Client> clients;
        Server server;

        List<Player> players;

        bool player2Show, player3Show, player4Show;
        int blind, pot, chipsInRound;
        int roundCount = 0;
        String moveMessage;
        bool gameStarted;
        bool clientsConnected;
        int clientsAvailable;
        const int raisePerRoundMax = 2;
        bool bettingRoundComplete;


        public Form1()
        {
            this.FormBorderStyle = FormBorderStyle.FixedSingle;
            this.DoubleBuffered = true;
            horizontalCard = new Bitmap("C:\\Users\\Andrew\\documents\\visual studio
2010\\Projects\\THEServer\\THEServer\\Card Images\\b2fh.png");
            verticalCard = new Bitmap("C:\\Users\\Andrew\\documents\\visual studio
2010\\Projects\\THEServer\\THEServer\\Card Images\\b2fv.png");
            InitializePoints();
            size = new Size(71, 96);
            size2 = new Size(96, 71);
            communityCards = new Card[5];
            bettingRoundComplete = false;
            player2Show = false;
```

```
    player3Show = false;
    player4Show = false;
    clientsConnected = false;
    clientsAvailable = 0;
    blind = 5;
    chipsInRound = 5;
    haveClients = false;
    gameStarted = false;
    players = new List<Player>();
    pot = 0;
    InitializeComponent();
}

private void button1_Click(object sender, EventArgs e)
{
    CreatePlayers(comboBox1.Text, comboBox2.Text, comboBox3.Text, textBox1.Text);

    button1.Dispose();
    comboBox1.Dispose();
    comboBox2.Dispose();
    comboBox3.Dispose();
    label1.Dispose();
    label2.Dispose();
    label3.Dispose();
    nameLabel.Dispose();
    textBox1.Dispose();

    Start();

}

public void Start()
{
    deck = new Deck();


    groupBox1.Visible = true;
    foldButton.Visible = true;
    callButton.Visible = true;
    raiseButton.Visible = true;
    label4.Visible = true;
    label5.Visible = true;

    Next();
}

private void InitializePoints()
{
    myPoints = new Point[2];
    player2Points = new Point[2];
    player3Points = new Point[2];
    player4Points = new Point[2];
    communityCardsPoints = new Point[5];

    myPoints[0] = new Point(480, 474);
    myPoints[1] = new Point(580, 474);

    player2Points[0] = new Point(20, 224);
    player2Points[1] = new Point(20, 324);

    player3Points[0] = new Point(480, 34);
```

```
            player3Points[1] = new Point(580, 34);

            player4Points[0] = new Point(990, 224);
            player4Points[1] = new Point(990, 324);

            communityCardsPoints[0] = new Point(330, 264);
            communityCardsPoints[1] = new Point(430, 264);
            communityCardsPoints[2] = new Point(530, 264);
            communityCardsPoints[3] = new Point(630, 264);
            communityCardsPoints[4] = new Point(730, 264);

        }

        private void BurnCard()
        {
            deck.GetNextCard();
        }

        private void PreFlop()
        {
            gameStarted = true;

            foreach (Player player in players)
            {
                player.SetBluff();
            }

            chipsInRound = 5;

            Deal();
            this.Refresh();

            if (haveClients)
                server.BroadcastState(players, pot, chipsInRound, communityCards,
String.Empty);

            TakeBets();
        }

        private void Flop()
        {
            communityCards[0] = deck.GetNextCard();
            communityCards[1] = deck.GetNextCard();
            communityCards[2] = deck.GetNextCard();
            foreach (Player player in players)
            {
                player.AddCard(communityCards[0]);
                player.AddCard(communityCards[1]);
                player.AddCard(communityCards[2]);
            }
            TakeBets();
        }

        private void Turn()
        {
            communityCards[3] = deck.GetNextCard();
            foreach (Player player in players)
            {
                player.AddCard(communityCards[3]);
            }
            TakeBets();
```

```
}

private void River()
{
    communityCards[4] = deck.GetNextCard();
    foreach (Player player in players)
    {
        player.AddCard(communityCards[4]);
    }
    TakeBets();
}

private void Showdown()
{
    player2Show = true; player3Show = true; player4Show = true;


    List<Player> pl = new List<Player>();
    if (players[0].StillIn) pl.Add(players[0]);
    if (players[1].StillIn) pl.Add(players[1]);
    if (players[2].StillIn) pl.Add(players[2]);
    if (players[3].StillIn) pl.Add(players[3]);
    Winner winner = new Winner(pl, pot);

    messageBox.Items.Add(winner.GetMessage());

    pot = 0;

    player2Show = false; player3Show = false; player4Show = false;
    CheckForDone();
    ClearHands();
}

public void RefreshNumbers()
{
    youChips.Text = players[0].Chips.ToString();
    player2Chips.Text = players[1].Chips.ToString();
    player3Chips.Text = players[2].Chips.ToString();
    player4Chips.Text = players[3].Chips.ToString();
}

public void RoundOver()
{
    roundCount++;
    if (roundCount == 5) roundCount = 0;
    players[0].RoundOver();
    players[1].RoundOver();
    players[2].RoundOver();
    players[3].RoundOver();
    chipsInRound = 0;
    fullCircle = false;
}

public void Next()
{
    first = true;
    switch (roundCount)
    {
        case 0:
            bettingRoundComplete = false;
            PreFlop();
```

```
                if (!players[0].StillIn) Next();
                break;
            case 1:
                bettingRoundComplete = false;
                Flop();
                if (!players[0].StillIn) Next();
                break;
            case 2:
                bettingRoundComplete = false;
                Turn();
                if (!players[0].StillIn) Next();
                break;
            case 3:
                bettingRoundComplete = false;
                River();
                if (!players[0].StillIn) Next();
                break;
            case 4:
                bettingRoundComplete = false;
                Showdown();
                RoundOver();
                Next();
                break;
        }
        blind = 5;
    }

    private void TakeBets()
    {
        if (!first)
        {
            bettingRoundComplete = true;

            if (players[0].StillIn && (players[0].ChipsInRound != chipsInRound &&
!players[0].AllIn))
                bettingRoundComplete = false;
            if (players[1].StillIn && (players[1].ChipsInRound != chipsInRound &&
!players[1].AllIn))
                bettingRoundComplete = false;
            if (players[2].StillIn && (players[2].ChipsInRound != chipsInRound &&
!players[2].AllIn))
                bettingRoundComplete = false;
            if (players[3].StillIn && (players[3].ChipsInRound != chipsInRound &&
!players[3].AllIn))
                bettingRoundComplete = false;

            if (bettingRoundComplete)
            {
                RoundOver();
                Next();
                return;
            }
        }
        else
            first = false;

        if (players.Where(p => p.StillIn).Count() == 1)
        {
            roundCount = 4; Next(); return;
        }
```

```
            if (players[1].StillIn)
            {
                Thread.Sleep(1000);
                players[1].TakeTurn(ref pot, ref chipsInRound, ref moveMessage, roundCount);
                messageBox.Items.Add(moveMessage);
                if (haveClients)
                    server.BroadcastState(players, pot, chipsInRound, communityCards,
moveMessage);
                RefreshNumbers();
                this.Refresh();

            }
            if (players[2].StillIn)
            {
                Thread.Sleep(1000);
                players[2].TakeTurn(ref pot, ref chipsInRound, ref moveMessage, roundCount);
                messageBox.Items.Add(moveMessage);
                if (haveClients)
                    server.BroadcastState(players, pot, chipsInRound, communityCards,
moveMessage);
                RefreshNumbers();
                this.Refresh();

            }
            if (players[3].StillIn)
            {
                Thread.Sleep(1000);
                players[3].TakeTurn(ref pot, ref chipsInRound, ref moveMessage, roundCount);
                messageBox.Items.Add(moveMessage);
                if (haveClients)
                    server.BroadcastState(players, pot, chipsInRound, communityCards,
moveMessage
                        );
                RefreshNumbers();
                this.Refresh();
            }

            if (players[0].StillIn)
            {
                if (!players[1].StillIn && !players[2].StillIn && !players[3].StillIn)
                { roundCount = 4; Next(); return; }

                if (players[0].ChipsInRound == chipsInRound)
                {
                    if (fullCircle)
                    {
                        RoundOver();
                        Next();
                        return;
                    }
                    else
                    {
                        fullCircle = true;
                    }
                }

                players[0].IsTurn = true;
                messageBox.Items.Add("your turn");
                this.Refresh();
            }
            else TakeBets();
```

```
        }

        private void CheckForDone()
        {
            //to account for folding
            players[0].StillIn = true;
            players[1].StillIn = true;
            players[2].StillIn = true;
            players[3].StillIn = true;
            if (players[0].Chips <= 0) { players[0].StillIn = false; this.Dispose(); }
            if (players[1].Chips <= 0) players[1].StillIn = false;
            if (players[2].Chips <= 0) players[2].StillIn = false;
            if (players[3].Chips <= 0) players[3].StillIn = false;
        }

        private void Deal()
        {
            deck.Shuffle();
            for (int i = 0; i < 2; i++)
            {
                if(players[0].StillIn)players[0].AddCard(deck.GetNextCard());
                if (players[1].StillIn) players[1].AddCard(deck.GetNextCard());
                if (players[2].StillIn) players[2].AddCard(deck.GetNextCard());
                if (players[3].StillIn) players[3].AddCard(deck.GetNextCard());
            }
        }

        private void ClearHands()
        {
            players[0].ClearHand();
            players[1].ClearHand();
            players[2].ClearHand();
            players[3].ClearHand();

            for (int i = 0; i < communityCards.Count(); i++)
            {
                communityCards[i] = null;
            }
            players[0].AllIn = false; players[1].AllIn = false; players[2].AllIn = false;
players[3].AllIn = false;
        }

        private void foldButton_Click(object sender, EventArgs e)
        {
            if (!players[0].IsTurn) return;
            players[0].Fold(ref moveMessage);
            if (haveClients)
                server.BroadcastState(players, pot, chipsInRound, communityCards, moveMessage);
            if (!bettingRoundComplete)
                TakeBets();
            else
                Next();
        }

        private void callButton_Click(object sender, EventArgs e)
        {
            if (!players[0].IsTurn) return;
            players[0].Call(chipsInRound - players[0].ChipsInRound, ref pot, ref moveMessage);
            if (haveClients)
                server.BroadcastState(players, pot, chipsInRound, communityCards, moveMessage);
```

```
            RefreshNumbers();
            this.Refresh();
            if (!bettingRoundComplete)
                TakeBets();
            else
                Next();
        }

        private void raiseButton_Click(object sender, EventArgs e)
        {
            if (!players[0].IsTurn || !players[0].CanRaise()) return;
            players[0].Raise(chipsInRound - players[0].ChipsInRound, ref pot,
Convert.ToInt16(raiseBox.Text), ref chipsInRound, ref moveMessage);
            chipsInRound = players[0].ChipsInRound;
            if (haveClients)
                server.BroadcastState(players, pot, chipsInRound, communityCards, moveMessage);

            RefreshNumbers();
            this.Refresh();
            if (!bettingRoundComplete)
                TakeBets();
            else
                Next();
        }

        public void CreatePlayers(String p2, String p3, String p4, String name)
        {
            Names names = new Names();
            Player you = new Player();
            players.Add(you);
            players[0].Name = (name == "")?names.GetName():name;
            names.GetName();

            clients = new List<Client>();

            if (p2 == "AI")
            {
                AI player = new AI();
                players.Add(player);
                players[1].Name = names.GetName();
            }
            else if (p2 == "Client")
            {
                Client player = new Client();
                players.Add(player);
                clients.Add(player);
                clientsAvailable++;
                player.SetOrder(2, 3, 0, 1);

            }
            players[1].IsSideWays = true;
            if (p3 == "AI")
            {
                AI player = new AI();
                players.Add(player);
                players[2].Name = names.GetName();
            }
            else if (p3 == "Client")
            {
                Client player = new Client();
                players.Add(player);
```

```
            clients.Add(player);
            clientsAvailable++;
            player.SetOrder(3, 0, 1, 2);
        }

        if (p4 == "AI")
        {
            AI player = new AI();
            players.Add(player);
            players[3].Name = names.GetName();
        }
        else if (p4 == "Client")
        {
            Client player = new Client();
            players.Add(player);
            clients.Add(player);
            clientsAvailable++;
            player.SetOrder(0, 1, 2, 3);
        }
        players[3].IsSideWays = true;
        SetNames();

        if(clientsAvailable != 0)
        {
            server = new Server(clients);

            haveClients = true;


            server.GetNames();
        }

        SetNames();
    }

    private void SetNames()
    {
        youNameLabel.Text = players[0].Name;
        player2NameLabel.Text = players[1].Name;
        player3NameLabel.Text = players[2].Name;
        player4NameLabel.Text = players[3].Name;
    }

    private void Form1_Paint(object sender, PaintEventArgs e)
    {
        int count = 0;
        Graphics g = e.Graphics;

        this.messageBox.TopIndex = this.messageBox.Items.Count - 1;




        label5.Text = pot.ToString();

        if (gameStarted)
        {

            if (players[0].IsTurn)
            {
```

```
                    if (players[0].ChipsInRound < chipsInRound)
                    {
                        callButton.Text = "Call " + (chipsInRound -
players[0].ChipsInRound).ToString();
                    }
                    else
                    {
                        callButton.Text = "Check";
                    }
                }

                foreach (Point p in communityCardsPoints)
                {
                    g.DrawImage((communityCards[count] != null) ? communityCards[count].Image :
verticalCard, new Rectangle(p, size));
                    count++;
                }

                g.DrawImage((players[0].Hand[0] != null) ? players[0].Image1 : verticalCard,
new Rectangle(myPoints[0], size));
                g.DrawImage((players[0].Hand[1] != null) ? players[0].Image2 : verticalCard,
new Rectangle(myPoints[1], size));


                if (players[1].StillIn)
                {
                    g.DrawImage((player2Show) ? players[1].Image1 : horizontalCard, new
Rectangle(player2Points[0], size2));
                    g.DrawImage((player2Show) ? players[1].Image2 : horizontalCard, new
Rectangle(player2Points[1], size2));
                }

                if (players[3].StillIn)
                {
                    g.DrawImage((player4Show) ? players[3].Image1 : horizontalCard, new
Rectangle(player4Points[0], size2));
                    g.DrawImage((player4Show) ? players[3].Image2 : horizontalCard, new
Rectangle(player4Points[1], size2));
                }


                if (players[2].StillIn)
                {
                    g.DrawImage((player3Show) ? players[2].Image1 : verticalCard, new
Rectangle(player3Points[0], size));
                    g.DrawImage((player3Show) ? players[2].Image2 : verticalCard, new
Rectangle(player3Points[1], size));
                }
            }
        }
    }
}
```

AI.cs

```
using System;

using System.Collections.Generic;
using System.Linq;
using System.Text;
using SharedAssemblies;

namespace THEServer
{
    [Serializable]
    class AI:Player
    {
        public Dictionary<int, OutPercentages> Odds { get; set; }

        public Dictionary<CardType, int> CardTypeRating { get; set; }

        public AI()
        {
            Odds = new Dictionary<int, OutPercentages>();
            CardTypeRating = new Dictionary<CardType, int>();
            SetOdds();
        }

        public override void SetBluff()
        {
            //to be called beginning of each game
            //10% chance of bluffing

            if (RandomNum(1, 10) == 1)
                Bluff = true;
        }

        public void SetOdds()
        {
            OutPercentages op0 = new OutPercentages(0, 0, 0);
            Odds[0] = op0;
            OutPercentages op1 = new OutPercentages(2.13, 2.17, 4.26);
            Odds[1] = op1;
            OutPercentages op2 = new OutPercentages(4.26,    4.35,  8.42);
            Odds[2] = op2;
            OutPercentages op3 = new OutPercentages(6.38,    6.52,  12.49);
            Odds[3] = op3;
            OutPercentages op4 = new OutPercentages(8.51,    8.70,  16.47);
            Odds[4] = op4;
            OutPercentages op5 = new OutPercentages(10.64,   10.87, 20.35);
            Odds[5] = op5;
            OutPercentages op6 = new OutPercentages(12.77,   13.04, 24.14);
            Odds[6] = op6;
            OutPercentages op7 = new OutPercentages(14.89,   15.22, 27.84);
            Odds[7] = op7;
            OutPercentages op8 = new OutPercentages(17.02,   17.39, 31.45);
            Odds[8] = op8;
            OutPercentages op9 = new OutPercentages(19.15,   19.57, 34.97);
            Odds[9] = op9;
            OutPercentages op10 = new OutPercentages(21.28,  21.74, 38.39);
            Odds[10] = op10;
            OutPercentages op11 = new OutPercentages(23.40,  23.91, 41.72);
            Odds[11] = op11;
```

```
            OutPercentages op12 = new OutPercentages(25.53,  26.09, 44.96);
            Odds[12] = op12;
            OutPercentages op13 = new OutPercentages(27.66,  28.26, 48.10);
            Odds[13] = op13;
            OutPercentages op14 = new OutPercentages(29.79,  30.43, 51.16);
            Odds[14] = op14;
            OutPercentages op15 = new OutPercentages(31.91,  32.61, 54.12);
            Odds[15] = op15;
            OutPercentages op16 = new OutPercentages(34.04,  34.78, 56.98);
            Odds[16] = op16;
            OutPercentages op17 = new OutPercentages(36.17,  36.96, 59.76);
            Odds[17] = op17;
            OutPercentages op18 = new OutPercentages(38.30,  39.13, 62.44);
            Odds[18] = op18;
            OutPercentages op19 = new OutPercentages(40.43,  41.30, 65.03);
            Odds[19] = op19;
            OutPercentages op20 = new OutPercentages(42.55,  43.48, 67.53);
            Odds[20] = op20;
            OutPercentages op21 = new OutPercentages(44.68,  45.65, 69.94);
            Odds[21] = op21;
            OutPercentages op22 = new OutPercentages(46.81,  47.83, 72.25);
            Odds[22] = op22;

            CardTypeRating[CardType.Ace] = 13;
            CardTypeRating[CardType.King] = 12;
            CardTypeRating[CardType.Queen] = 11;
            CardTypeRating[CardType.Jack] = 10;
            CardTypeRating[CardType.Ten] = 9;
            CardTypeRating[CardType.Nine] = 8;
            CardTypeRating[CardType.Eight] = 7;
            CardTypeRating[CardType.Seven] = 6;
            CardTypeRating[CardType.Six] = 5;
            CardTypeRating[CardType.Five] = 4;
            CardTypeRating[CardType.Four] = 3;
            CardTypeRating[CardType.Three] = 2;
            CardTypeRating[CardType.Two] = 1;
        }

        private int RandomNum(int min, int max)
        {
            Random random = new Random();
            return random.Next(min, max);
        }


        //remember to pass in betting round integer
        public override void TakeTurn(ref int pot, ref int chipsInRound, ref String Message,
int bettingRound)
        {

            //postflop
            //get best current hand (rank by int) 0 - 100 scale

            int callAmount = chipsInRound - ChipsInRound;

            if (bettingRound == 0)
            {
                //preflop
                bool cardsSame = false;
                bool suitsSame = false;
```

```
            if (Hand[0].CardType == Hand[1].CardType)
                cardsSame = true;

            if (Hand[0].Suit == Hand[1].Suit)
                suitsSame = true;

            int card1Rank = CardTypeRating[Hand[0].CardType];
            int card2Rank = CardTypeRating[Hand[1].CardType];

            if (cardsSame || suitsSame || card1Rank + card2Rank > 19)
            {
                //75% chance of raising
                int num = RandomNum(1, 4);

                if (num > 1)
                {
                    //raise


                    int betAmount = RandomNum(Chips / 50, Chips / 10);

                    //raise betAmount

                    if (callAmount == 0)
                        Raise(callAmount, ref pot, betAmount, ref chipsInRound, ref
Message);
                    else
                    {
                        betAmount = betAmount - callAmount;

                        if (betAmount < 0)
                            Call(callAmount, ref pot, ref Message);
                        else
                            Raise(callAmount, ref pot, betAmount, ref chipsInRound, ref
Message);
                    }
                }
                else
                {
                    if (callAmount == 0)
                        Check(ref Message);
                    else
                    {
                        if (callAmount > Chips / 15)
                            Fold(ref Message);
                        else
                            Call(callAmount, ref pot, ref Message);
                    }
                }
            }
            else
            {

                if (Bluff)
                {
                    int betAmount = RandomNum(Chips / 50, Chips / 10);

                    //raise betAmount
                    if (callAmount == 0)
                        Raise(callAmount, ref pot, betAmount, ref chipsInRound, ref
Message);
```

```
                    else
                    {
                        betAmount = betAmount - callAmount;

                        if (betAmount < 0)
                            Call(callAmount, ref pot, ref Message);
                        else
                            Raise(callAmount, ref pot, betAmount, ref chipsInRound, ref
Message);
                    }

                }
                else
                {
                    if (callAmount == 0)
                        Check(ref Message);
                    else
                    {
                        if (callAmount > Chips / 15)
                            Fold(ref Message);
                        else
                            Call(callAmount, ref pot, ref Message);
                    }
                }
            }

        }
        else if (bettingRound == 3)
        {
            //have all cards no need to calculate outs
            //bet solely on comparing best hand to pot odds

            THEServer.Hand h = new THEServer.Hand(Hand);
            Result r = h.GetHandRank();
            int rank = r.HandRank;

            switch (rank)
            {
                case 1:
                    rank = 100;
                    break;
                case 2:
                    rank = 77 + GetHighCardAmount(r.HighCards); //+ amount < 11 for high
cards
                    break;
                case 3:
                    rank = 66 + GetHighCardAmount(r.HighCards);
                    break;
                case 4:
                    rank = 55 + GetHighCardAmount(r.HighCards);
                    break;
                case 5:
                    rank = 44 + GetHighCardAmount(r.HighCards);
                    break;
                case 6:
                    rank = 33 + GetHighCardAmount(r.HighCards);
                    break;
                case 7:
                    rank = 22 + GetHighCardAmount(r.HighCards);
                    break;
                case 8:
```

```
                rank = 11 + GetHighCardAmount(r.HighCards);
                break;
            case 9:
                rank = 0 + GetHighCardAmount(r.HighCards);
                break;
            default:
                break;
        }
        //use rank as percent chance of raising


        if (callAmount == 0)
        {
            //decide whether to raise or call

            int rnum = RandomNum(1, 100);

            if (rnum <= rank)
            {
                //raise

                double myNum = rank / 100.0;
                myNum *= Chips;

                int betAmount = RandomNum(Chips / 50, Convert.ToInt16(myNum));

                Raise(callAmount, ref pot, betAmount, ref chipsInRound, ref Message);

            }
            else if (Bluff)
            {

                int betAmount = RandomNum(Chips / 50, Chips / 10);
                Raise(callAmount, ref pot, betAmount, ref chipsInRound, ref Message);
            }
            else
            {
                Check(ref Message);
            }

        }
        else
        {

            int potOdds = 1 / ( pot / callAmount );

            if ((rank / 100) > potOdds)
            {
                //either call or raise
                int rnum = RandomNum(1, 4);

                if ((rank / 100) > (potOdds * rnum))
                {
                    //raise
                    double myNum = rank / 100.0;
                    myNum *= Chips;

                    int betAmount = RandomNum(Chips / 50, Convert.ToInt16(myNum));

                    Raise(callAmount, ref pot, betAmount, ref chipsInRound, ref
Message);
```

```
                    }
                    else if (Bluff)
                    {
                        int betAmount = RandomNum(Chips / 50, Chips / 10);
                        Raise(callAmount, ref pot, betAmount, ref chipsInRound, ref
Message);
                    }
                    else
                    {
                        Call(callAmount, ref pot, ref Message);
                    }
                }
                else if (Bluff)
                {
                    int betAmount = RandomNum(Chips / 50, Chips / 10);
                    Raise(callAmount, ref pot, betAmount, ref chipsInRound, ref Message);
                }
                else
                {
                    //chance to still call
                    int rnum = RandomNum(1, 3);

                    if (rnum == 1)
                    {
                        Call(callAmount, ref pot, ref Message);
                    }
                    else
                    {
                        Fold(ref Message);
                    }

                }


            }
        }
        else
        {
            THEServer.Hand h = new THEServer.Hand(Hand);
            Result r = h.GetHandRank();
            int rank = r.HandRank;

            switch (rank)
            {
                case 1:
                    rank = 100;
                    break;
                case 2:
                    rank = 77 + GetHighCardAmount(r.HighCards); //+ amount < 11 for high
cards
                    break;
                case 3:
                    rank = 66 + GetHighCardAmount(r.HighCards);
                    break;
                case 4:
                    rank = 55 + GetHighCardAmount(r.HighCards);
                    break;
                case 5:
                    rank = 44 + GetHighCardAmount(r.HighCards);
                    break;
                case 6:
```

```
            rank = 33 + GetHighCardAmount(r.HighCards);
            break;
        case 7:
            rank = 22 + GetHighCardAmount(r.HighCards);
            break;
        case 8:
            rank = 11 + GetHighCardAmount(r.HighCards);
            break;
        case 9:
            rank = 0 + GetHighCardAmount(r.HighCards);
            break;
        default:
            break;
    }
    //determine outs percentage 0 - 100 scale
    double oprank = 0;


    if (bettingRound == 1)
        oprank = Odds[h.DetermineOuts()].forTheTurnAndRiver;
    else if (bettingRound == 2)
        oprank = Odds[h.DetermineOuts()].forTheRiver;


    if (rank >= 44)
    {
        if (oprank < rank)
            oprank = rank;
    }

    rank = (rank + Convert.ToInt16(oprank));

    if (callAmount == 0)
    {
        //decide whether to raise or call

        int rnum = RandomNum(1, 100);

        if (rnum <= rank)
        {
            //raise
            double myNum = rank / 100.0;
            if (myNum > 1.0)
                myNum = 1.0;
            myNum *= Chips;

            int betAmount = RandomNum(Chips / 50, Convert.ToInt16(myNum));

            Raise(callAmount, ref pot, betAmount, ref chipsInRound, ref Message);

        }
        else if (Bluff)
        {
            int betAmount = RandomNum(Chips / 50, Chips / 10);
            Raise(callAmount, ref pot, betAmount, ref chipsInRound, ref Message);
        }
        else
        {
            Check(ref Message);
        }
```

```
            }
            else
            {
                //compare to pot odds
                int potOdds = 1 / (pot / callAmount);

                if ((rank / 100) > potOdds)
                {
                    //either call or raise
                    int rnum = RandomNum(1, 4);

                    if ((rank / 100) > (potOdds * rnum))
                    {
                        //raise
                        int betAmount = RandomNum(Chips / 50, Chips * (rank / 100));

                        Raise(callAmount, ref pot, betAmount, ref chipsInRound, ref
Message);
                    }
                    else if (Bluff)
                    {
                        int betAmount = RandomNum(Chips / 50, Chips / 10);
                        Raise(callAmount, ref pot, betAmount, ref chipsInRound, ref
Message);
                    }
                    else
                    {
                        Call(callAmount, ref pot, ref Message);
                    }
                }
                else if (Bluff)
                {
                    int betAmount = RandomNum(Chips / 50, Chips / 10);
                    Raise(callAmount, ref pot, betAmount, ref chipsInRound, ref Message);
                }
                else
                {
                    //chance to still call
                    int rnum = RandomNum(1, 3);

                    if (rnum == 1)
                    {
                        Call(callAmount, ref pot, ref Message);
                    }
                    else
                    {
                        Fold(ref Message);
                    }

                }

            }

        }

    }

    int GetHighCardAmount(List<CardType> c)
    {
        if (c.Exists(b => b == CardType.Ace))
            return 10;
```

```
            CardType ct = c.Max();
            switch (ct)
            {
                case CardType.King:
                    return 9;
                case CardType.Queen:
                    return 8;
                case CardType.Jack:
                    return 7;
                case CardType.Ten:
                    return 6;
                case CardType.Nine:
                    return 5;
                case CardType.Eight:
                    return 4;
                case CardType.Seven:
                    return 3;
                case CardType.Six:
                    return 2;
                case CardType.Five:
                    return 1;
                default:
                    return 0;
            }
        }
    }
}
```

Client.cs

```
using System;

using System.Collections.Generic;
using System.Linq;
using System.Net.Sockets;
using System.Text;
using SharedAssemblies;

namespace THEServer
{
    [Serializable]
    class Client : Player
    {

        public TcpClient client { get; set; }

        public int[] order { get; set; }
        public int spot { get; set; }

        public Client()
        {

            order = new int[3];

        }

        public void SetOrder(int a, int b, int c, int d)
        {
            order[0] = a;
            order[1] = b;
            order[2] = c;
            spot = d;
        }

        public void GetName()
        {
            NetworkStream clientStream = client.GetStream();

            String responseData = String.Empty;

            while (true)
            {
                byte[] bytes = new byte[256];
                int bytesRec = clientStream.Read(bytes, 0, 256);

                responseData += Encoding.ASCII.GetString(bytes, 0, bytesRec);

                if (responseData.IndexOf("<end>") > -1)
                {
                    break;
                }
            }

            // remove <end>
            responseData = responseData.Replace("<end>", "");

            Name = responseData;
        }
```

```
        public override void TakeTurn(ref int pot, ref int chipsInRound, ref string
moveMessage, int bettingRound)
        {
            NetworkStream clientStream = client.GetStream();
            //send isturn to client

            ASCIIEncoding encoder = new ASCIIEncoding();
            //byte[] toEncodeAsBytes = System.Text.ASCIIEncoding.ASCII.GetBytes("isturn<end>");
            //string returnValue = System.Convert.ToBase64String(toEncodeAsBytes);

            byte[] buffer = encoder.GetBytes("isturn<end>");

            clientStream.Write(buffer, 0, buffer.Length);
            clientStream.Flush();

            String responseData = String.Empty;

            while (true)
            {
                byte[] bytes = new byte[256];

                int bytesRec = clientStream.Read(bytes, 0, 256);

                responseData += Encoding.ASCII.GetString(bytes, 0, bytesRec);

                if (responseData.IndexOf("<end>") > -1)
                {
                    break;
                }
            }
            // remove <end>
            responseData = responseData.Replace("<end>", "");

            switch (responseData)
            {
                case "fold":
                    base.Fold(ref moveMessage);
                    break;
                case "call":
                    if (ChipsInRound < chipsInRound)
                        base.Call(chipsInRound - ChipsInRound, ref pot, ref moveMessage);
                    else
                        base.Check(ref moveMessage);
                    break;
                default:
                    //break up raise and int

                    string[] words = responseData.Split(' ');

                    if (words[0] == "raise")
                    {
                        base.Raise(chipsInRound - ChipsInRound, ref pot,
Convert.ToInt16(words[1]), ref chipsInRound, ref moveMessage);
                    }
                    break;
            }
        }
    }
}
```

Deck.cs

```
using System;

using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Drawing;
using SharedAssemblies;

namespace THEServer
{
    class Deck
    {
        public Card[] deck { get; set; }
        public int currentPosition;
        private static Random random = new Random();

        public Deck()
        {
            currentPosition = 0;
            deck = new Card[52];
            Suit[] suits = { Suit.Clubs, Suit.Diamonds, Suit.Hearts, Suit.Spades };
            CardType[] cardTypes = {CardType.Ace, CardType.Two, CardType.Three, CardType.Four,
CardType.Five, CardType.Six,
                                    CardType.Seven, CardType.Eight, CardType.Nine,
CardType.Ten, CardType.Jack, CardType.Queen, CardType.King};
            int count = 0;
            foreach (Suit suit in suits)
            {
                foreach (CardType cardType in cardTypes)
                {
                    deck[count] = new Card();
                    deck[count].Suit = suit;
                    deck[count].CardType = cardType;
                    char c = (char)((int)suit);
                    char c2 = (char)((int)cardType);
                    String s;
                    if(c2 != ':') s = "C:\\Users\\Andrew\\documents\\visual studio
2010\\Projects\\THEServer\\THEServer\\Card Images\\" +c.ToString() + c2.ToString() + ".png";
                    else s = "C:\\Users\\Andrew\\documents\\visual studio
2010\\Projects\\THEServer\\THEServer\\Card Images\\" + c.ToString() + "10.png";
                    deck[count].Image = new Bitmap(s);
                    count++;
                }
            }
        }

        public void Shuffle()
        {
            currentPosition = 0;

            if (deck.Count() > 1)
            {
                for (int i = deck.Count() - 1; i >= 0; i--)
                {
                    Card tmp = deck[i];
                    int randomIndex = random.Next(i + 1);

                    //Swap elements
```

```
                deck[i] = deck[randomIndex];
                deck[randomIndex] = tmp;
            }
        }
    }

    public Card GetNextCard()
    {
        int cp = currentPosition;
        currentPosition++;
        if (cp < 52) return deck[cp];
        else return null;
    }
  }
}
```

Hand.cs

```
using System;

using System.Collections.Generic;
using System.Linq;
using System.Text;
using SharedAssemblies;

namespace THEServer
{
    class Hand
    {
        //Card one, two, three, four, five, six, seven;
        private int handRank;
        List<CardType> highCards;
        List<Card> Outs;
        List<Card> hand;
        Suit[] suits;
        CardType[] cardTypes;

        public Hand(List<Card> cards)
        {
            Outs = new List<Card>();
            hand = cards;
            suits = new Suit[] { Suit.Diamonds, Suit.Clubs, Suit.Hearts, Suit.Spades };
            cardTypes = new CardType[] { CardType.Two, CardType.Three, CardType.Four,
CardType.Five,
                                         CardType.Six, CardType.Seven, CardType.Eight,
CardType.Nine,
                                         CardType.Ten, CardType.Jack, CardType.Queen,
CardType.King,
                                         CardType.Ace };
            OrderHandByRank();
            highCards = new List<CardType>();
        }

        private void OrderHandByRank()
        {
            if (hand.Exists(c => c.CardType == CardType.Ace))
            {
                List<Card> aces = new List<Card>();
                for (int i = hand.Count() - 1; i >= 0; i--)
                {
                    if (hand[i].CardType == CardType.Ace)
                    {
                        aces.Add(hand[i]);
                        hand.Remove(hand[i]);
                    }
                }
                hand = hand.OrderByDescending(c => c.CardType).ToList();
                for (int i = 0; i < aces.Count(); i++)
                {
                    hand.Insert(i, aces[i]);
                }
            }
            else
            {
                hand = hand.OrderByDescending(c => c.CardType).ToList();
            }
```

```
        }

        public int DetermineOuts()
        {
            //check for nearly having a straight

            List<Card> myList = hand.Distinct<Card>(new MyComparer()).ToList();

            myList = myList.OrderByDescending(o => o.CardType).ToList();
            //Queue<Card> optimalHand = new Queue<Card>();
            int streak = 1;
            bool missingCard = false;
            CardType missingCardType = CardType.Null;
            //optimalHand.Enqueue(myList.First());
            CardType lastCard = myList[0].CardType;

            for (int i = 1; i < myList.Count(); i++)
            {
                if (myList[i].CardType == lastCard - 1)
                {
                    streak++;
                    //optimalHand.Enqueue(myList[i]);
                    lastCard = myList[i].CardType;
                }
                else if (!missingCard)
                {

                    streak++;
                    missingCard = true;
                    missingCardType = myList[i - 1].CardType - 1;
                }
                else
                {
                    streak = 1;
                    missingCard = false;
                    //optimalHand.Clear();
                    //optimalHand.Enqueue(myList[i]);
                }

                if (streak == 5)
                {
                    foreach (Suit suit in suits)
                    {
                        Card card = new Card();
                        card.CardType = missingCardType;
                        card.Suit = suit;
                        Outs.Add(card);
                    }
                }
            }

            bool rs = true;
            missingCard = false;
            //optimalHand.Clear();
            missingCardType = CardType.Null;
            if (myList.Find(c => c.CardType == CardType.Ace) == null) { missingCard = true;
missingCardType = CardType.Ace; }

            if (myList.Find(c => c.CardType == CardType.King) == null && missingCard) rs =
false;
            else if (myList.Find(c => c.CardType == CardType.King) == null) { missingCard =
```

```
true; missingCardType = CardType.King; }

            if (rs && myList.Find(c => c.CardType == CardType.Queen) == null && missingCard) rs
= false;
            else if (myList.Find(c => c.CardType == CardType.Queen) == null) { missingCard =
true; missingCardType = CardType.Queen; }

            if (rs && myList.Find(c => c.CardType == CardType.Jack) == null && missingCard) rs
= false;
            else if (myList.Find(c => c.CardType == CardType.Jack) == null) { missingCard =
true; missingCardType = CardType.Jack; }

            if (rs && myList.Find(c => c.CardType == CardType.Ten) == null && missingCard) rs =
false;
            else if (myList.Find(c => c.CardType == CardType.Ten) == null) { missingCard =
true; missingCardType = CardType.Ten; }

            if (rs)
            {
                 foreach (Suit suit in suits)
                 {
                     Card card = new Card();
                     card.CardType = missingCardType;
                     card.Suit = suit;
                     Outs.Add(card);
                 }
             }

            //check for nearly having a four of a kind

            for (int i = 0; i < hand.Count(); i++)
            {
                CardType cardType = hand[i].CardType;
                if (hand.Where(h => h.CardType == cardType).Count() == 3)
                {
                    List<Card> cs = hand.Where(h => h.CardType == cardType).ToList();
                    Suit suit = suits.Where(s => cs.Find(c => c.Suit == s) == null).First();


                    if (!Outs.Exists(c => c.CardType == cardType && c.Suit == suit))
                    {
                        Card c2 = new Card();
                        c2.CardType = cardType;
                        c2.Suit = suit;

                        Outs.Add(c2);
                    }
                }
            }

            //check for nearly having fullhouse

            for (int i = 0; i < hand.Count(); i++)
            {
                CardType cardType = hand[i].CardType;
                if (hand.Where(h => h.CardType == cardType).Count() == 2)
                {
                    for (int j = 0; j < hand.Count(); j++)
                    {
                        CardType cardType2 = hand[j].CardType;
                        if (hand.Where(h => h.CardType == cardType2).Count() == 2 && cardType2
```

```
!= cardType)
                    {
                        List<Card> cs = hand.Where(h => h.CardType == cardType).ToList();
                        List<Card> cs2 = hand.Where(h => h.CardType == cardType2).ToList();
                        List<Suit> suits1 = suits.Where(s => cs.Find(c => c.Suit == s) ==
null).ToList();
                        List<Suit> suits2 = suits.Where(s => cs2.Find(c => c.Suit == s) ==
null).ToList();

                        foreach (Suit suit in suits1)
                        {
                            if (!Outs.Exists(c => c.CardType == cardType && c.Suit ==
suit))
                            {
                                Card c = new Card();
                                c.CardType = cardType;
                                c.Suit = suit;

                                Outs.Add(c);
                            }
                        }

                        foreach (Suit suit in suits2)
                        {
                            if (!Outs.Exists(c => c.CardType == cardType2 && c.Suit ==
suit))
                            {
                                Card c = new Card();
                                c.CardType = cardType2;
                                c.Suit = suit;

                                Outs.Add(c);
                            }
                        }

                    }
                }
            }
        }

        //check for nearly having a flush

        for (int i = 0; i < hand.Count(); i++)
        {
            Suit suit = hand[i].Suit;
            List<Card> hcs;
            if ((hcs = hand.Where(h => h.Suit == suit).ToList()).Count() == 4)
            {
                List<Card> c = hand.Where(h => h.Suit == suit).ToList();
                List<CardType> cts = cardTypes.Where(ct => c.Find(b => b.CardType == ct) ==
null).ToList();

                foreach (CardType cardType in cts)
                {
                    if (!Outs.Exists(ca => ca.CardType == cardType && ca.Suit == suit))
                    {
                        Card card = new Card();
                        card.CardType = cardType;
                        card.Suit = suit;
                        Outs.Add(card);
                    }
```

```
                }

                //List<Suit> suits1 = suits.Where(s => cs.Find(c => c.Suit == s) ==
null).ToList();

            }
        }

        return Outs.Count();
    }

    public Result GetHandRank()
    {
        if (CheckForStraightFlush()) return new Result(highCards, 1);
        else if (CheckForFourOfAKind()) return new Result(highCards, 2);
        else if (CheckForFullHouse()) return new Result(highCards, 3);
        else if (CheckForFlush()) return new Result(highCards, 4);
        else if (CheckForStraight()) return new Result(highCards, 5);
        else if (CheckForThreeOfAKind()) return new Result(highCards, 6);
        else if (CheckForTwoPair()) return new Result(highCards, 7);
        else if (CheckForOnePair()) return new Result(highCards, 8);
        else { GetHighCard(); return new Result(highCards, 9); }
    }

    public bool CheckForStraightFlush()
    {
        List<Card> myList = hand.Distinct<Card>(new MyComparer()).ToList();

        if (myList.Count() < 5) return false;

        myList = myList.OrderByDescending(o => o.CardType).ToList();
        Queue<Card> optimalHand = new Queue<Card>();
        int streak = 1;
        optimalHand.Enqueue(myList.First());
        for (int i = 1; i < myList.Count(); i++)
        {
            if (myList[i].CardType == myList[i - 1].CardType - 1)
            {
                streak++;
                optimalHand.Enqueue(myList[i]);
            }
            else
            {
                streak = 1;
                optimalHand.Clear();
                optimalHand.Enqueue(myList[i]);
            }

            if (streak == 5)
            {
                foreach (Suit suit in suits)
                {
                    bool isFlush = true;
                    foreach (Card c in optimalHand)
                    {
                        if (!hand.Exists(x => x.CardType == c.CardType && x.Suit == suit))
{ isFlush = false; break; }
                    }
                    if (isFlush) { highCards.Add(optimalHand.First().CardType);
highCards.Add(CardType.Null); return true; }
                }
```

```
                optimalHand.Clear();
                i = i - 3;
                optimalHand.Enqueue(myList[i]);
                streak = 1;
            }
        }

        //Check for royal flush (Ace high)


        bool rf = true;
        optimalHand.Clear();
        if (myList.Find(c => c.CardType == CardType.Ace) == null) rf = false;
        else if (myList.Find(c => c.CardType == CardType.King) == null) rf = false;
        else if (myList.Find(c => c.CardType == CardType.Queen) == null) rf = false;
        else if (myList.Find(c => c.CardType == CardType.Jack) == null) rf = false;
        else if (myList.Find(c => c.CardType == CardType.Ten) == null) rf = false;

        if (rf)
        {
            CardType[] cards = new CardType[] { CardType.Ace, CardType.King,
CardType.Queen, CardType.Jack, CardType.Ten };
            foreach (Suit suit in suits)
            {
                bool isRoyalFlush = true;
                foreach (CardType c in cards)
                {
                    if (!hand.Exists(x => x.CardType == c && x.Suit == suit)) {
isRoyalFlush = false; break; }
                }
                if (isRoyalFlush) { highCards.Add(CardType.Ace);
highCards.Add(CardType.Null); return true; }
            }
        }

        return false;
    }

    public bool CheckForFourOfAKind()
    {
        for (int i = 0; i < hand.Count(); i++)
        {
            CardType cardType = hand[i].CardType;
            if (hand.Where(h => h.CardType == cardType).Count() == 4)
            {
                highCards.Add(cardType);
                if (hand.Exists(c => c.CardType == CardType.Ace) && cardType !=
CardType.Ace)
                {
                    highCards.Add(CardType.Ace);
                }
                else
                {
                    highCards.Add(hand.OrderByDescending(c => c.CardType).Where(c =>
c.CardType != cardType).First().CardType);
                }
                return true;
            }
        }
        return false;
```

```
        }

        public bool CheckForFullHouse()
        {
            for (int i = 0; i < hand.Count(); i++)
            {
                CardType cardType = hand[i].CardType;
                if (hand.Where(h => h.CardType == cardType).Count() == 3)
                {
                    for (int j = 0; j < hand.Count(); j++)
                    {
                        CardType cardType2 = hand[j].CardType;
                        if (hand.Where(h => h.CardType == cardType2).Count() == 2 && cardType2
!= cardType)
                        {
                            highCards.Add(cardType);
                            highCards.Add(cardType2);
                            highCards.Add(CardType.Null);
                            return true;
                        }
                    }
                }
            }

            return false;
        }

        public bool CheckForFlush()
        {
            for (int i = 0; i < hand.Count(); i++)
            {
                Suit suit = hand[i].Suit;
                List<Card> hcs;
                if ((hcs = hand.Where(h => h.Suit == suit).ToList()).Count() == 5)
                {
                    if (hcs.Exists(c => c.CardType == CardType.Ace))
                    {
                        Card tempCard = hcs.Where(c => c.CardType == CardType.Ace).First();
                        highCards.Add(tempCard.CardType);
                        hcs.Remove(tempCard);
                    }
                    hcs = hcs.OrderByDescending(c => c.CardType).ToList();
                    foreach (Card card in hcs)
                    {
                        highCards.Add(card.CardType);
                    }
                    return true;
                }
            }
            return false;
        }

        public bool CheckForStraight()
        {
            List<Card> myList = hand.Distinct<Card>(new MyComparer()).ToList();

            if (myList.Count() < 5) return false;

            myList = myList.OrderByDescending(o => o.CardType).ToList();
            Queue<Card> optimalHand = new Queue<Card>();
            int streak = 1;
```

```
            optimalHand.Enqueue(myList.First());
            for (int i = 1; i < myList.Count(); i++)
            {
                if (myList[i].CardType == myList[i - 1].CardType - 1)
                {
                    streak++;
                    optimalHand.Enqueue(myList[i]);
                }
                else
                {
                    streak = 1;
                    optimalHand.Clear();
                    optimalHand.Enqueue(myList[i]);
                }

                if (streak == 5) { highCards.Add(optimalHand.First().CardType);
highCards.Add(CardType.Null); return true; }
            }


            bool rs = true;
            optimalHand.Clear();
            if (myList.Find(c => c.CardType == CardType.Ace) == null) rs = false;
            else if (myList.Find(c => c.CardType == CardType.King) == null) rs = false;
            else if (myList.Find(c => c.CardType == CardType.Queen) == null) rs = false;
            else if (myList.Find(c => c.CardType == CardType.Jack) == null) rs = false;
            else if (myList.Find(c => c.CardType == CardType.Ten) == null) rs = false;

            if (rs)
            {
                highCards.Add(CardType.Ace);
                highCards.Add(CardType.Null);
                return true;
            }
            return false;
        }

        public bool CheckForThreeOfAKind()
        {
            for (int i = 0; i < hand.Count(); i++)
            {
                CardType cardType = hand[i].CardType;
                if (hand.Where(h => h.CardType == cardType).Count() == 3)
                {
                    highCards.Add(cardType);
                    if (hand.Exists(c => c.CardType == CardType.Ace) && cardType !=
CardType.Ace)
                    {
                        highCards.Add(CardType.Ace);
                        highCards.Add(hand.OrderByDescending(c => c.CardType).Where(c =>
c.CardType != cardType).First().CardType);
                        highCards.Add(CardType.Null);
                    }
                    else
                    {
                        CardType cardType2 = hand.OrderByDescending(c => c.CardType).Where(c =>
c.CardType != cardType).First().CardType;
                        highCards.Add(cardType2);
                        highCards.Add(hand.OrderByDescending(c => c.CardType).Where(c =>
c.CardType != cardType && c.CardType != cardType2).First().CardType);
                        highCards.Add(CardType.Null);
```

```
                }
                return true;
            }
        }
        return false;
    }

    public bool CheckForTwoPair()
    {
        for (int i = 0; i < hand.Count(); i++)
        {
            CardType cardType = hand[i].CardType;
            if (hand.Where(h => h.CardType == cardType).Count() == 2)
            {
                for (int j = 0; j < hand.Count(); j++)
                {
                    CardType cardType2 = hand[j].CardType;
                    if (hand.Where(h => h.CardType == cardType2).Count() == 2 && cardType2
!= cardType)
                    {
                        if (cardType == CardType.Ace || cardType > cardType2)
                        {
                            highCards.Add(cardType);
                            highCards.Add(cardType2);
                        }
                        else if (cardType2 == CardType.Ace || cardType2 > cardType)
                        {
                            highCards.Add(cardType2);
                            highCards.Add(cardType);
                        }

                        if (cardType != CardType.Ace && cardType2 != CardType.Ace &&
hand.Exists(c => c.CardType == CardType.Ace))
                        {
                            highCards.Add(CardType.Ace);
                        }
                        else
                        {
                            highCards.Add(hand.OrderByDescending(c => c.CardType).Where(c
=> c.CardType != cardType && c.CardType != cardType2).First().CardType);
                        }
                        highCards.Add(CardType.Null);
                        return true;
                    }
                }
            }
        }
        return false;
    }

    public bool CheckForOnePair()
    {
        for (int i = 0; i < hand.Count(); i++)
        {
            CardType cardType = hand[i].CardType;
            if (hand.Where(h => h.CardType == cardType).Count() == 2)
            {
                highCards.Add(cardType);

                if (cardType != CardType.Ace && hand.Exists(c => c.CardType ==
CardType.Ace))
```

```
                    {
                        highCards.Add(CardType.Ace);
                    }
                    else
                    {
                        highCards.Add(hand.OrderByDescending(c => c.CardType).Where(c =>
c.CardType != cardType).First().CardType);
                    }
                    highCards.Add(CardType.Null);
                    return true;
                }
            }
            return false;
        }

        public void GetHighCard()
        {
            if (hand.Exists(c => c.CardType == CardType.Ace))
            {
                highCards.Add(CardType.Ace);
            }
            else
            {
                highCards.Add(hand.OrderByDescending(c => c.CardType).First().CardType);
            }
            highCards.Add(CardType.Null);
        }
    }

    public class MyComparer : IEqualityComparer<Card>
    {
        #region IEqualityComparer<Card> Members
        bool IEqualityComparer<Card>.Equals(Card x, Card y)
        {
            // Check whether the compared objects reference the same data.
            if (Object.ReferenceEquals(x, y))
                return true;

            // Check whether any of the compared objects is null.
            if (Object.ReferenceEquals(x, null) || Object.ReferenceEquals(y, null))
                return false;

            return x.CardType == y.CardType;
        }

        int IEqualityComparer<Card>.GetHashCode(Card obj)
        {
            return obj.CardType.GetHashCode();
        }
        #endregion
    }
}
```

MyListBox.cs

```
using System;

using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace THEServer
{
    using System;
    using System.ComponentModel;
    using System.Windows.Forms;

    public class MyListBox : ListBox
    {
        private bool mShowScroll;
        protected override CreateParams CreateParams
        {
            get
            {
                CreateParams cp = base.CreateParams;
                if (!mShowScroll) cp.Style &= ~0x200000;  // Turn off WS_VSCROLL
                return cp;
            }
        }
        [DefaultValue(false)]
        public bool ShowScrollbar
        {
            get { return mShowScroll; }
            set
            {
                if (value == mShowScroll) return;
                mShowScroll = value;
                if (this.Handle != IntPtr.Zero) RecreateHandle();
            }
        }
    }
}
```

Names.cs

```
using System;

using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace THEServer
{
    class Names
    {
        public List<String> names;
        public Names()
        {
            names = new List<String>(new String[] { "Amy", "Bob", "Andrew", "Gary", "Valerie",
"Austin", "Tim", "Jamie", "Bryan", "Tom", "Jimmy", "Jen", "Hank", "Theodore", "Robert", "Jose",
"Sarah", "Hallie", "Rebecca", "Sam", "Sean", "Danny", "Ian", "Harry", "Aaron", "Erin", "Kayla",
"Derek", "Lebron", "Leon", "Jeff", "James", "Jason", "Fred", "Donny", "Ben", "Tony", "Dave",
"Ronny", "Oliver", "Olivia", "Karen", "Carrie", "Kasey", "Keisha", "Kevin", "Leo", "Adam",
"Addy", "Levi", "Henry", "Davey", "Devon", "Darcy", "Deandre", "Stephen", "Steve", "Will",
"Wade", "Bill", "Betty", "Andre", "Chris", "Janet", "Ron", "Melvin", "Marty", "Matt", "Josh",
"Jesse", "Mike", "Michael", "Mandy", "Mindy", "Dell", "Jed", "Drew", "Woody", "Martin",
"Whoopi", "Chuck", "Bruce", "Richard", "Sidney", "Marcus", "Stephen", "Alshon", "Jadaveon"});
        }

        public String GetName()
        {
            int index = RandomNumber(0, names.Count() - 1);

            String name = names[index];
            names.Remove(names[index]);

            return name;
        }

        private int RandomNumber(int min, int max)
        {
            Random random = new Random();
            return random.Next(min, max);
        }
    }
}
```

Out.cs

```
using System;

using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace THEServer
{
    [Serializable]
    class OutPercentages
    {
        public double forTheTurn;
        public double forTheRiver;
        public double forTheTurnAndRiver;

        public OutPercentages(double a, double b, double c)
        {
            forTheTurn = a;
            forTheRiver = b;
            forTheTurnAndRiver = c;
        }

    }
}
```

Player.cs

```
using System;

using System.Collections.Generic;
using System.Drawing;
using System.Linq;
using System.Text;

namespace SharedAssemblies
{
    [Serializable]
    public class Player
    {
        public List<Card> Hand { get; set; }
        public bool StillIn { get; set; }
        public bool IsTurn { get; set; }
        public bool AllIn { get; set; }
        public bool IsSideWays { get; set; }
        public String Name { get; set; }
        public int MatchesWon { get; set; }
        public int Chips { get; set; }
        public int ChipsInRound { get; set; }
        private int RaisesPerRound { get; set; }
        public String Message { get; set; }
        public Bitmap Image1 { get; set; }
        public Bitmap Image2 { get; set; }
        public bool Bluff { get; set; }
        public Player()
        {
            Hand = new List<Card>();

            Bluff = false;
            StillIn = true;
            AllIn = false;
            RaisesPerRound = 0;
            MatchesWon = 0;
            IsTurn = false;
            IsSideWays = false;
            Chips = 200;
            ChipsInRound = 0;
        }

        public virtual void SetBluff() { }

        public virtual void TakeTurn(ref int pot, ref int chipsInRound, ref String moveMessage,
int bettingRound) { }

        public void RoundOver()
        {
            RaisesPerRound = 0;
            ChipsInRound = 0;
        }

        public bool CanRaise()
        {
            if (RaisesPerRound >= 2)
                return false;
            else
                return true;
```

```
        }

        public void AddCard(Card card)
        {
            Hand.Add(card);
            if (Hand.Count() == 2)
            {
                if (IsSideWays)
                {
                    Hand[0].Image.RotateFlip(RotateFlipType.Rotate90FlipNone);
                    Hand[1].Image.RotateFlip(RotateFlipType.Rotate90FlipNone);
                }
                Image1 = Hand[0].Image;
                Image2 = Hand[1].Image;
            }
        }

        public void ClearHand()
        {
            Bluff = false;
            if (IsSideWays)
            {
                //Hand[0].Image.RotateFlip(RotateFlipType.Rotate270FlipNone);
                //Hand[1].Image.RotateFlip(RotateFlipType.Rotate270FlipNone);
                Image1.RotateFlip(RotateFlipType.Rotate270FlipNone);
                Image2.RotateFlip(RotateFlipType.Rotate270FlipNone);

            }
            Hand.Clear();
        }

        public virtual void Call(int betAmount, ref int pot, ref String message)
        {
            if (Chips <= betAmount)
            {
                pot += Chips;
                ChipsInRound += Chips;
                Chips = 0;
                AllIn = true;
                message = Name + " is all in";
            }
            else
            {
                Chips -= betAmount;
                ChipsInRound += betAmount;
                pot += betAmount;
                message = Name + " calls";
            }

            IsTurn = false;
        }

        public virtual void Check(ref String message)
        {
            message = Name + " checks";
            IsTurn = false;
        }

        public virtual void Raise(int betAmount, ref int pot, int raiseAmount, ref int
chipsInRound, ref String message)
        {
```

```
            if (!CanRaise() || raiseAmount == 0)
            {
                Call(betAmount, ref pot, ref message);
                return;
            }
            betAmount += raiseAmount;
            if (betAmount > Chips)
            {
                betAmount = Chips;

                if (raiseAmount > betAmount)
                    chipsInRound += raiseAmount - betAmount;
            }
            Chips -= betAmount;
            ChipsInRound += betAmount;
            chipsInRound += raiseAmount;
            pot += betAmount;
            message = Name + " raises " + raiseAmount;
            if (Chips == 0) { AllIn = true; message = Name + " is all in"; }
            RaisesPerRound++;
            IsTurn = false;
        }

        public virtual void Fold(ref String message)
        {
            message = Name + " folds";
            StillIn = false;
            IsTurn = false;
        }
    }
}
```

Result.cs

```
using System;

using System.Collections.Generic;
using System.Linq;
using System.Text;
using SharedAssemblies;

namespace THEServer
{
    class Result
    {
        public Result(List<CardType> HighCards, int HandRank)
        {
            this.HandRank = HandRank;
            this.HighCards = HighCards;
        }

        public int HandRank { get; set; }
        public List<CardType> HighCards { get; set; }
    }
}
```

Server.cs

```
using System;

using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Net.Sockets;
using System.Threading;
using System.Net;
using System.Xml.Serialization;
using System.IO;
using System.Runtime.Serialization.Formatters.Binary;
using SharedAssemblies;

namespace THEServer
{
    class Server
    {
        private TcpListener tcpListener;
        private Thread listenThread;
        List<Client> clients;

        public Server(List<Client> cs)
        {
            this.clients = cs;
            this.tcpListener = new TcpListener(IPAddress.Any, 3000);

            this.tcpListener.Start();

            //blocks until a client has connected to the server
            foreach(Client client in cs)
            {
                TcpClient c = this.tcpListener.AcceptTcpClient();
                client.client = c;
            }
        }

        public void GetNames()
        {
            foreach (Client client in clients)
            {
                client.GetName();
            }
        }

        public void BroadcastState(List<Player> players, int pot, int chipsInRound, Card[]
communityCards, String message)
        {
            foreach (Client client in clients.Where(c => c.StillIn))
            {
                BoardState state = new BoardState();
                //set properties unique to client
                state.MoveMessage = message;
                state.Chips = client.Chips;
                state.Players = new ShellPlayer[3];
                for(int i = 0; i < client.order.Count(); i++)
                {
                    ShellPlayer sp = new ShellPlayer();
                    sp.StillIn = players[client.order[i]].StillIn;
```

```
                sp.Name = players[client.order[i]].Name;
                sp.Chips = players[client.order[i]].Chips;
                sp.ChipsInRound = players[client.order[i]].ChipsInRound;
                sp.Hand = new List<ShellCard>();
                ShellCard s1 = new ShellCard();
                ShellCard s2 = new ShellCard();

                s1.CardType = players[client.order[i]].Hand[0].CardType;
                s1.Suit = players[client.order[i]].Hand[0].Suit;

                s2.CardType = players[client.order[i]].Hand[1].CardType;
                s2.Suit = players[client.order[i]].Hand[1].Suit;
                sp.Hand.Add(s1);
                sp.Hand.Add(s2);
                state.Players[i] = sp;
            }

            state.CommunityCards = new ShellCard[5];
            for(int i = 0; i < communityCards.Count(); i++)
            {
                if (communityCards[i] != null)
                {
                    ShellCard c = new ShellCard();
                    c.CardType = communityCards[i].CardType;
                    c.Suit = communityCards[i].Suit;

                    state.CommunityCards[i] = c;
                }
            }

            state.PotAmount = pot;

            state.PlayerCards = new ShellCard[2];
            ShellCard c1 = new ShellCard();
            ShellCard c2 = new ShellCard();
            c1.CardType = client.Hand[0].CardType;
            c1.Suit = client.Hand[0].Suit;

            c2.CardType = client.Hand[1].CardType;
            c2.Suit = client.Hand[1].Suit;


            state.PlayerCards[0] = c1;
            state.PlayerCards[1] = c2;

            if (client.ChipsInRound < chipsInRound)
            {
                state.CallCheckMessage = "Call " + (chipsInRound -
client.ChipsInRound).ToString();
            }
            else
            {
                state.CallCheckMessage = "Check";
            }


            XmlSerializer serializer = new XmlSerializer(typeof(BoardState));

            using (StringWriter writer = new StringWriter())
            {
                serializer.Serialize(writer, state);
```

```
                byte[] buffer = Encoding.ASCII.GetBytes(writer.ToString() + "<end>");

                NetworkStream clientStream = client.client.GetStream();

                clientStream.Write(buffer, 0, buffer.Length);

                clientStream.Flush();
            }
        }
    }
}
```

Winner.cs

```
using System;

using System.Collections.Generic;
using System.Linq;
using System.Text;
using SharedAssemblies;

namespace THEServer
{
    class Winner
    {
        public List<Player> Players { get; set; }
        private List<Result> Results;
        private String message;
        private int pot;
        public Winner(List<Player> players, int pot)
        {
            this.Players = players;
            Results = new List<Result>();
            this.pot = pot;
            DetermineWinner();
        }

        private void DetermineWinner()
        {
            if (Players.Count() == 1)
            {
                Players[0].Chips += pot;
                message = Players[0].Name + " wins";
                return;
            }

            foreach (Player player in Players)
            {
                Hand hand = new Hand(player.Hand);
                Result result = hand.GetHandRank();
                Results.Add(result);
            }

            int bestHand = 10;
            Dictionary<Result, int> winners = new Dictionary<Result, int>();
            for (int i = 0; i < Results.Count(); i++)
            {
                if (Results[i].HandRank < bestHand)
                {
                    winners.Clear();
                    winners[Results[i]] = i;
                    bestHand = Results[i].HandRank;
                }
                else if (Results[i].HandRank == bestHand)
                {
                    winners[Results[i]] = i;
                }
            }

            if (winners.Count > 1)
            {
                //multiple indexes where result handrank is lowest
```

```
                //tiebreaker

                int index = 0;
                while (true)
                {
                    if (index == 5 || winners.First().Key.HighCards[index] == CardType.Null)
                    {
                        List<Player> ps = new List<Player>();
                        foreach (KeyValuePair<Result, int> kvp in winners)
                        {
                            ps.Add(Players[kvp.Value]);
                        }
                        Tie(ps);
                        break;
                    }
                    CardType w;
                    //if (index == 0)
                    //{
                        w = (winners.Keys.ToList().Exists(c => c.HighCards[index] ==
CardType.Ace)
                                ? CardType.Ace
                                : winners.Max(m => m.Key.HighCards[index]));
                    //}
                    //else
                    //{
                    //    w = winners.Max(m => m.Key.HighCards[index]);
                    //}
                    winners = winners.Where(p => p.Key.HighCards[index] == w).ToDictionary(v =>
v.Key, v => v.Value);
                    if (winners.Count() == 1)
                    {
                        Player winner = Players[winners.First().Value];
                        winner.Chips += pot;
                        CreateMessage(bestHand, winner.Name);
                        break;
                    }

                    index++;
                }
            }
            else
            {
                Player winner = Players[winners.First().Value];
                winner.Chips += pot;
                CreateMessage(bestHand, winner.Name);
            }
        }

        public void Tie(List<Player> p)
        {
            int share = pot / p.Count();
            for (int i = 0; i < p.Count(); i++)
            {
                p[i].Chips += share;
                message += p[i].Name + " ";
                if ((i + 2) == p.Count()) message += "and ";
            }
            message += "split the pot";
        }

        public void CreateMessage(int handRank, String playerName)
```

```
        {
            switch (handRank)
            {
                case 1:
                    message = playerName + " wins with a straight flush";
                    break;
                case 2:
                    message = playerName + " wins with a four of a kind";
                    break;
                case 3:
                    message = playerName + " wins with a full house";
                    break;
                case 4:
                    message = playerName + " wins with a flush";
                    break;
                case 5:
                    message = playerName + " wins with a straight";
                    break;
                case 6:
                    message = playerName + " wins with a three of a kind";
                    break;
                case 7:
                    message = playerName + " wins with a two pair";
                    break;
                case 8:
                    message = playerName + " wins with a one pair";
                    break;
                case 9:
                    message = playerName + " wins with a high card";
                    break;
            }
        }

        public String GetMessage()
        {
            return message;
        }
    }
}
```

THEClient

Form1.cs

```
using System;

using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using System.Threading;
using System.Net.Sockets;
using System.Net;
using System.IO;
using System.Xml.Serialization;
using System.Runtime.Serialization.Formatters.Binary;
using SharedAssemblies;

namespace THEClient
{
    public partial class Form1 : Form
    {
        bool player2Show, player3Show, player4Show;
        Bitmap horizontalCard, verticalCard;
        Size size, size2;
        //Deck deck;
        Point[] myPoints, player2Points, player3Points, player4Points, communityCardsPoints;
        TcpClient client;
        IPEndPoint serverEndPoint;
        NetworkStream clientStream;
        bool gameStarted;
        bool isTurn;
        bool player2StillIn, player3StillIn, player4StillIn;
        ShellCard[] communityCards;
        ShellCard[] playerCards;
        Bitmap card1, card2;

        public Form1()
        {
            this.DoubleBuffered = true;
            this.FormBorderStyle = FormBorderStyle.FixedSingle;
            horizontalCard = new Bitmap("C:\\Users\\Andrew\\documents\\visual studio
2010\\Projects\\THEServer\\THEClient\\Card Images\\b2fh.png");
            verticalCard = new Bitmap("C:\\Users\\Andrew\\documents\\visual studio
2010\\Projects\\THEServer\\THEClient\\Card Images\\b2fv.png");
            InitializePoints();
            size = new Size(71, 96);
            size2 = new Size(96, 71);
            communityCards = new ShellCard[5];
            playerCards = new ShellCard[2];

            player2Show = false;
            player3Show = false;
            player4Show = false;
            player2StillIn = true;
            player3StillIn = true;
            player4StillIn = true;

            gameStarted = false;
```

```
            isTurn = false;




            InitializeComponent();
        }

        private void button1_Click(object sender, EventArgs e)
        {

            button1.Dispose();
            nameLabel.Dispose();

            label1.Dispose();

            label2.Visible = true;

            foldButton.Visible = true;
            callButton.Visible = true;
            raiseButton.Visible = true;

            client = new TcpClient();

            serverEndPoint = new IPEndPoint(IPAddress.Parse(textBox2.Text), 3000);

            youNameLabel.Text = textBox1.Text;

            client.Connect(serverEndPoint);

            SendName();

            textBox1.Dispose();
            textBox2.Dispose();

            ProcessMessage(ReceiveMessage());
        }

        private void SendName()
        {
            ASCIIEncoding encoder = new ASCIIEncoding();
            byte[] buffer = encoder.GetBytes(youNameLabel.Text + "<end>");
            clientStream = client.GetStream();

            clientStream.Write(buffer, 0, buffer.Length);
            clientStream.Flush();

            isTurn = false;
        }

        private Bitmap GetImage(ShellCard card)
        {
            char c = (char)((int)card.Suit);
            char c2 = (char)((int)card.CardType);
            String s;
            if(c2 != ':') s = "C:\\Users\\Andrew\\documents\\visual studio
2010\\Projects\\THEServer\\THEClient\\Card Images\\" +c.ToString() + c2.ToString() + ".png";
            else s = "C:\\Users\\Andrew\\documents\\visual studio
2010\\Projects\\THEServer\\THEClient\\Card Images\\" + c.ToString() + "10.png";
            Bitmap image = new Bitmap(s);
```

```
    return image;
}

private void UpdateInterface(BoardState state)
{
    for (int i = 0; i < state.CommunityCards.Count(); i++)
    {
        communityCards[i] = state.CommunityCards[i];
    }

    for (int i = 0; i < state.PlayerCards.Count(); i++)
    {
        playerCards[i] = state.PlayerCards[i];
    }

    player2StillIn = state.Players[0].StillIn;
    player3StillIn = state.Players[1].StillIn;
    player4StillIn = state.Players[2].StillIn;
    youChips.Text = state.Chips.ToString();

    player2NameLabel.Text = state.Players[0].Name;
    player2Chips.Text = state.Players[0].Chips.ToString();
    player3NameLabel.Text = state.Players[1].Name;
    player3Chips.Text = state.Players[1].Chips.ToString();
    player4NameLabel.Text = state.Players[2].Name;
    player4Chips.Text = state.Players[2].Chips.ToString();

    if(state.MoveMessage != String.Empty)
        messageBox.Items.Add(state.MoveMessage);

    callButton.Text = state.CallCheckMessage;
    potLabel3.Text = state.PotAmount.ToString();


    this.Refresh();
}

private void ProcessMessage(String message)
{
    switch (message)
    {
        case "isturn":
            isTurn = true;
            messageBox.Items.Add("your turn");
            break;
        default:
            gameStarted = true;
            groupBox1.Visible = true;


            var serializer = new XmlSerializer(typeof(BoardState));
            BoardState result;

            using (TextReader reader = new StringReader(message))
            {
               result = (BoardState)serializer.Deserialize(reader);

               UpdateInterface(result);
            }
            ProcessMessage(ReceiveMessage());
```

```
            break;
        }


    }

    private String ReceiveMessage()
    {
        // String to store the response ASCII representation.
        String responseData = String.Empty;

        clientStream = client.GetStream();

        while (true)
        {
            byte[] bytes = new byte[256];

            int bytesRec = clientStream.Read(bytes, 0, 256);



            responseData += Encoding.ASCII.GetString(bytes, 0, bytesRec);


            if (responseData.IndexOf("<end>") > -1)
            {
                break;
            }
        }
        clientStream.Flush();

        // remove <end>
        responseData = responseData.Replace("<end>", "");

        return responseData;

    }

    private void InitializePoints()
    {
        myPoints = new Point[2];
        player2Points = new Point[2];
        player3Points = new Point[2];
        player4Points = new Point[2];
        communityCardsPoints = new Point[5];

        myPoints[0] = new Point(480, 474);
        myPoints[1] = new Point(580, 474);

        player2Points[0] = new Point(20, 224);
        player2Points[1] = new Point(20, 324);

        player3Points[0] = new Point(480, 34);
        player3Points[1] = new Point(580, 34);

        player4Points[0] = new Point(990, 224);
        player4Points[1] = new Point(990, 324);

        communityCardsPoints[0] = new Point(330, 264);
        communityCardsPoints[1] = new Point(430, 264);
        communityCardsPoints[2] = new Point(530, 264);
```

```
        communityCardsPoints[3] = new Point(630, 264);
        communityCardsPoints[4] = new Point(730, 264);

    }

    private void foldButton_Click(object sender, EventArgs e)
    {
        if (isTurn)
        {
            ASCIIEncoding encoder = new ASCIIEncoding();
            byte[] buffer = encoder.GetBytes("fold<end>");

            clientStream.Write(buffer, 0, buffer.Length);
            clientStream.Flush();

            isTurn = false;
            ProcessMessage(ReceiveMessage());

        }
    }

    private void callButton_Click(object sender, EventArgs e)
    {
        if (isTurn)
        {
            ASCIIEncoding encoder = new ASCIIEncoding();
            byte[] buffer = encoder.GetBytes("call<end>");

            clientStream.Write(buffer, 0, buffer.Length);
            clientStream.Flush();

            isTurn = false;
            ProcessMessage(ReceiveMessage());

        }

    }

    private void raiseButton_Click(object sender, EventArgs e)
    {
        if (isTurn)
        {
            ASCIIEncoding encoder = new ASCIIEncoding();
            byte[] buffer = encoder.GetBytes("raise " + raiseBox.Text + "<end>");

            clientStream.Write(buffer, 0, buffer.Length);
            clientStream.Flush();

            isTurn = false;
            ProcessMessage(ReceiveMessage());

        }
    }

    private void Form1_Paint(object sender, PaintEventArgs e)
    {
        int count = 0;
        Graphics g = e.Graphics;

        this.messageBox.TopIndex = this.messageBox.Items.Count - 1;
```

```
            if (gameStarted)
            {
                foreach (Point p in communityCardsPoints)
                {
                    g.DrawImage((communityCards[count] != null) ?
GetImage(communityCards[count]) : verticalCard, new Rectangle(p, size));
                    count++;
                }


                g.DrawImage(GetImage(playerCards[0]), new Rectangle(myPoints[0], size));
                g.DrawImage(GetImage(playerCards[1]), new Rectangle(myPoints[1], size));


                if (player2StillIn)
                {
                    g.DrawImage(horizontalCard, new Rectangle(player2Points[0], size2));
                    g.DrawImage(horizontalCard, new Rectangle(player2Points[1], size2));
                }

                if (player4StillIn)
                {
                    g.DrawImage(horizontalCard, new Rectangle(player4Points[0], size2));
                    g.DrawImage(horizontalCard, new Rectangle(player4Points[1], size2));
                }


                if (player3StillIn)
                {
                    g.DrawImage(verticalCard, new Rectangle(player3Points[0], size));
                    g.DrawImage(verticalCard, new Rectangle(player3Points[1], size));
                }
            }
        }
    }
}
```

SharedAssemblies

BoardState.cs

```
using System;

using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SharedAssemblies
{

        [Serializable]
        public class BoardState
        {
            public ShellCard[] CommunityCards { get; set; }
            public ShellCard[] PlayerCards { get; set; }
            public ShellPlayer[] Players { get; set; }
            public int PotAmount { get; set; }
            public String CallCheckMessage { get; set; }
            public String MoveMessage { get; set; }
            public int Chips { get; set; }

            public BoardState()
            {

            }
        }

}
```

Card.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Drawing;

namespace SharedAssemblies
{
    [Serializable]
    public enum Suit { Hearts = 104, Diamonds = 100, Spades = 115, Clubs = 99 };
    [Serializable]
    public enum CardType { Null, Ace = 49, Two = 50, Three = 51, Four = 52, Five = 53, Six =
54, Seven = 55, Eight = 56, Nine = 57, Ten = 58, Jack = 106, Queen = 113, King = 107 };
    [Serializable]
    public class Card
    {
        public Suit Suit { get; set; }
        public CardType CardType { get; set; }
        public Bitmap Image { get; set; }
    }
}
```

ShellCard.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace SharedAssemblies
{

        [Serializable]
        public class ShellCard
        {
            public Suit Suit { get; set; }
            public CardType CardType { get; set; }

        }

}
```

ShellPlayer.cs

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Linq;
using System.Text;

namespace SharedAssemblies
{
    [Serializable]
    public class ShellPlayer
    {
        public List<ShellCard> Hand { get; set; }
        public bool StillIn { get; set; }
        public int Chips { get; set; }
        public int ChipsInRound { get; set; }
        public String Name { get; set; }
        public ShellPlayer()
        {

        }

    }
}
```